

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 9 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50 //
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMa)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMa m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89 //
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // FLOPs = 2 × M × K = 2 × 40962 ≈ 33 MFLOPs
102 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
103 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
104 //
105 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
106 //
107 // =====
108 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
109 // =====
110 // 面试要点:
111 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
112 //   memory
113 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
114 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
115 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
116 //
117 #include <algorithm>
118 #include <cstring>
119 #include <cuda_fp16.h>
120 #include <cuda_runtime.h>
121 #include <float.h>
122 #include <stdio.h>
123 #include <stdlib.h>
124 #include <math.h>
125 #include <cublas_v2.h>

```

```

126 #include <cuda.h>
127 #include <cuda/barrier>
128
129 #define WARP_SIZE 32
130 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
131 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
132 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
133
134 // =====
135 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
136 // =====
137
138 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
139 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
140 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
141 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
142 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
143 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
144 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
145 //
146 // 初始: 每个 lane 持有自己的值 v0..v7
147 // lane:  0    1    2    3    4    5    6    7
148 // val:  v0   v1   v2   v3   v4   v5   v6   v7
149 //
150 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
151 //
152 //      ┌──────────┐
153 // 对:   (0,4) (1,5) (2,6) (3,7)
154 //
155 // lane:  0    1    2    3    4    5    6    7
156 // val:  v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
157 //
158 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
159 //
160 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
161 // 对:   (0,2) (1,3) (4,6) (5,7)
162 //      ─── 前4个一组 ───      ─── 后4个一组 ───
163 //
164 // lane:  0    1    2    3    4    5    6    7 (每lane持有前一轮2个值的和再加本轮配对)
165 // val:   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$ 
166 //      = v0+v2+v4+v6 ... (逐步归约)
167 //
168 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
169 //
170 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
171 // 对:   (0,1) (2,3) (4,5) (6,7)
172 //
173 // lane:  0    1    2    3    4    5    6    7
174 // val:   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
175 //
176 // mask=0: 循环终止, 归约完成。
177 //
178 // 关键性质:
179 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
180 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
181 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
182 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
183 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
184 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
185
186 template <const int kWarpSize = WARP_SIZE, typename T = float>
187 __device__ __forceinline__ T warp_reduce_sum(T val) {
188 #pragma unroll
189     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
190         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
191     }
192     return val;
193 }

```

```

189 }
190
191 // Warp Reduce Max — generic
192 template <const int kWarpSize = WARP_SIZE, typename T = float>
193 __device__ __forceinline__ T warp_reduce_max(T val) {
194 #pragma unroll
195     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
196         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
197     }
198     return val;
199 }
200
201 // =====
202 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
203 // =====
204
205 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
206 // 两级归约流程:
207 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
208 // 2. warp leader (lane=0) 写入 shared memory
209 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
210 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
211 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
212 template <const int NUM_THREADS = 256>
213 __device__ float block_reduce_sum(float val) {
214     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
215     int warp = threadIdx.x / WARP_SIZE;
216     int lane = threadIdx.x % WARP_SIZE;
217     __shared__ float shared[NUM_WARPS];
218
219     float value = warp_reduce_sum<WARP_SIZE>(val);
220     if (lane == 0)
221         shared[warp] = value;
222     __syncthreads();
223     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
224     value = warp_reduce_sum<NUM_WARPS>(value);
225     // 关键: broadcast 结果到所有线程, 后续用 result
226     // 做除法等操作时所有线程都能拿到
227     value = __shfl_sync(0xffffffff, value, 0, 32);
228     return value;
229 }
230
231 // Block Reduce Max — FP32 (增强版, 带 broadcast)
232 template <const int NUM_THREADS = 256>
233 __device__ float block_reduce_max(float val) {
234     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
235     int warp = threadIdx.x / WARP_SIZE;
236     int lane = threadIdx.x % WARP_SIZE;
237     __shared__ float shared[NUM_WARPS];
238
239     float value = warp_reduce_max<WARP_SIZE>(val);
240     if (lane == 0)
241         shared[warp] = value;
242     __syncthreads();
243     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
244     value = warp_reduce_max<NUM_WARPS>(value);
245     value = __shfl_sync(0xffffffff, value, 0, 32);
246     return value;
247 }
248
249 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
250 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
251 // 跨 block 求和的常见模式, 适合 N 较大时使用;

```

```

252 // Grid: ((N + 255) / 256, 1, 1)
253 // Block: (256, 1, 1)
254 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
255 template <const int NUM_THREADS = 256>
256 __global__ void block_reduce_all(float *a, float *y, int N) {
257     int tid = threadIdx.x;
258     int idx = blockIdx.x * NUM_THREADS + tid;
259     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
260     __shared__ float shared[NUM_WARPS];
261
262     float val = (idx < N) ? a[idx] : 0.0f;
263     int warp = tid / WARP_SIZE;
264     int lane = tid % WARP_SIZE;
265
266     val = warp_reduce_sum<WARP_SIZE>(val);
267     if (lane == 0)
268         shared[warp] = val;
269     __syncthreads();
270
271     val = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
272     if (warp == 0)
273         val = warp_reduce_sum<NUM_WARPS>(val);
274     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
275         atomicAdd(y, val);
276 }
277
278 // ---- Dot Product: y = sum(a[i] * b[i]) ----
279 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
280 // Grid: ((N + 255) / 256, 1, 1)
281 // Block: (256, 1, 1)
282 // source: LeetCUDA/kernels/dot-product/dot_product.cu
283 template <const int NUM_THREADS = 256>
284 __global__ void dot(float *a, float *b, float *y, int N) {
285     int tid = threadIdx.x;
286     int idx = blockIdx.x * NUM_THREADS + tid;
287     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
288     __shared__ float shared[NUM_WARPS];
289
290     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
291     int warp = tid / WARP_SIZE;
292     int lane = tid % WARP_SIZE;
293
294     prod = warp_reduce_sum<WARP_SIZE>(prod);
295     if (lane == 0)
296         shared[warp] = prod;
297     __syncthreads();
298
299     prod = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
300     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
301         prod = warp_reduce_sum<NUM_WARPS>(prod);
302     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
303         atomicAdd(y, prod);
304 }
305
306 // Dot Product + float4
307 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素，float4向量化后每个线程处理4元素
308 // Block: (64, 1, 1), 256/4=64
309 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
310 // source: LeetCUDA/kernels/dot-product/dot_product.cu
311 template <const int NUM_THREADS = 256 / 4>
312 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
313     int tid = threadIdx.x;
314     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;

```

```

315 constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
316 __shared__ float shared[NUM_WARPS];
317
318 float4 reg_a = FLOAT4(a[idx]);
319 float4 reg_b = FLOAT4(b[idx]);
320 float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
321                          reg_a.z * reg_b.z + reg_a.w * reg_b.w)
322                  : 0.0f;
323 int warp = tid / WARP_SIZE;
324 int lane = tid % WARP_SIZE;
325
326 prod = warp_reduce_sum<WARP_SIZE>(prod);
327 if (lane == 0)
328     shared[warp] = prod;
329 __syncthreads();
330
331 prod = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
332 if (warp == 0)
333     prod = warp_reduce_sum<NUM_WARPS>(prod);
334 if (tid == 0)
335     atomicAdd(y, prod);
336 }
337
338
339 // =====
340 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
341 // =====
342 // 面试要点:
343 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
344 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
345 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
346
347 // ---- ReLU: y = max(0, x) ----
348 // Grid: ((N + 255) / 256, 1, 1)
349 // Block: (256, 1, 1)
350 // source: LeetCUDA/kernels/relu/relu.cu
351 __global__ void relu(float *x, float *y, int N) {
352     int idx = blockIdx.x * blockDim.x + threadIdx.x;
353     if (idx < N)
354         y[idx] = fmaxf(0.0f, x[idx]);
355 }
356
357 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
358 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
359 // Grid: ((N + 255) / 256, 1, 1)
360 // Block: (64, 1, 1)
361 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
362 // source: LeetCUDA/kernels/relu/relu.cu
363 __global__ void relu_vec4(float *x, float *y, int N) {
364     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
365     if (idx < N) {
366         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
367         float4 reg_y;
368         reg_y.x = fmaxf(0.0f, reg_x.x);
369         reg_y.y = fmaxf(0.0f, reg_x.y);
370         reg_y.z = fmaxf(0.0f, reg_x.z);
371         reg_y.w = fmaxf(0.0f, reg_x.w);
372         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
373     }
374 }
375
376 // ---- Elementwise Add: c = a + b ----
377 // Grid: ((N + 255) / 256, 1, 1)

```

```

378 // Block: (256, 1, 1)
379 // source: LeetCUDA/kernels/elementwise/elementwise.cu
380 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
381     int idx = blockIdx.x * blockDim.x + threadIdx.x;
382     if (idx < N)
383         c[idx] = a[idx] + b[idx];
384 }
385
386 // Elementwise Add + float4 向量化
387 // Grid: ((N + 255) / 256, 1, 1)
388 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
389 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
390 // source: LeetCUDA/kernels/elementwise/elementwise.cu
391 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
392     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
393     if ((idx + 3) < N) {
394         float4 reg_a = FLOAT4(a[idx]);
395         float4 reg_b = FLOAT4(b[idx]);
396         float4 reg_c;
397         reg_c.x = reg_a.x + reg_b.x;
398         reg_c.y = reg_a.y + reg_b.y;
399         reg_c.z = reg_a.z + reg_b.z;
400         reg_c.w = reg_a.w + reg_b.w;
401         FLOAT4(c[idx]) = reg_c;
402     } else if (idx < N) {
403         for (int i = 0; (idx + i) < N; i++) {
404             c[idx + i] = a[idx + i] + b[idx + i];
405         }
406     }
407 }
408
409 // ---- Histogram: y[a[i]]++ ----
410 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
411 // Grid: ((N + 255) / 256, 1, 1)
412 // Block: (256, 1, 1)
413 // source: LeetCUDA/kernels/histogram/histogram.cu
414 __global__ void histogram(int *a, int *y, int N) {
415     int idx = blockIdx.x * blockDim.x + threadIdx.x;
416     if (idx < N)
417         atomicAdd(&(y[a[idx]]), 1);
418 }
419
420 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
421 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
422 //
423 // 面试要点:
424 // - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
425 //   每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
426 // - 数学公式 (5 步推导):
427 //   Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
428 //       L_max = max(LSE_1, LSE_2)
429 //   Step 2 — 还原指数权重 (un-normalized):
430 //       w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
431 //   Step 3 — 归一化为混合比例:
432 //       alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
433 //       满足 alpha + beta = 1
434 //   Step 4 — 逐元素加权合并:
435 //       O = alpha * O_1 + beta * O_2
436 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention 惯例一致,
437 //   lse[head_idx][token_idx])
438 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
439 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
440 // - inf LSE → -inf: 空 attention 段 (causal mask 等导致全部 score

```



```

441 // 为 -inf) 的 LSE 可能为 +inf, 替换后  $\exp(-\text{inf} - L_{\text{max}}) = 0$ ,
442 // 该段权重退化为 0
443 // - 128-bit 向量化: uint4 = 4 × float, 每线程处理 4 个输出元素,
444 // pack load → FMA → pack store 一条流水线
445 // - AI ≈ 3 FLOP / 20 bytes ≈ 0.15 FLOPS/Byte → severely memory-bound
446 //
447 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
448 // pack_size = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
449 // threads_per_head = head_size / 4 = 2
450 // total_threads = num_tokens * num_heads * threads_per_head = 8
451 //
452 // global_idx:      0      1      2      3      4      5      6      7
453 // token_head_idx:  0      0      1      1      2      2      3      3
454 // (第几个 (token,head) 对, 行优先)
455 // pack_idx:        0      1      0      1      0      1      0      1
456 // (该 (token,head) 对内第几个 pack)
457 // token_idx:       0      0      0      0      1      1      1      1
458 // (token_head_idx / num_heads, token 变化慢)
459 // head_idx:        0      0      1      1      0      0      1      1
460 // (token_head_idx % num_heads, head 变化快)
461 // pack_offset:     0      4      0      4      0      4      0      4
462 // (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
463 // head_offset:     0      0      8      8      16     16     24     24
464 // (token_idx * num_heads * head_size + head_idx * head_size,
465 // 该 (token,head) 对在 output 展平数组中的起始偏移)
466 //
467 // Grid: ((total_threads + 127) / 128, 1, 1)
468 // Block: (128, 1, 1)
469 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
470 __global__ void merge_attn_states(
471     float *output,           // [num_tokens, num_heads, head_size]
472     const float *prefix_output, // [num_tokens, num_heads, head_size]
473     const float *prefix_lse,   // [num_heads, num_tokens]
474     const float *suffix_output, // [num_tokens, num_heads, head_size]
475     const float *suffix_lse,   // [num_heads, num_tokens]
476     int num_tokens, int num_heads, int head_size) {
477
478     constexpr int NUM_THREADS = 128;
479     constexpr int PACK_SIZE = 16 / sizeof(float); // 4 floats = 128-bit
480     using pack_t = uint4;
481
482     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
483     const int threads_per_head = head_size / PACK_SIZE;
484     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照NUM_THREADS=128
485     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
486     const int total_threads = num_tokens * num_heads * threads_per_head;
487
488     const int global_idx = blockIdx.x * NUM_THREADS + threadIdx.x;
489     if (global_idx >= total_threads)
490         return;
491
492     // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
493     // token_head_idx: 第几个 (token, head) 对, 行优先展平
494     const int token_head_idx = global_idx / threads_per_head;
495     // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
496     const int pack_idx = global_idx % threads_per_head;
497
498     // token_head_idx 分解为 (token_idx, head_idx)
499     const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
500     const int head_idx = token_head_idx % num_heads; // head 变化快 (内维)
501
502     // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
503     const int pack_offset = pack_idx * PACK_SIZE;

```

```

504 // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
505 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
506
507 // 定位到当前 (token, head) 的输出段起始
508 const float *prefix_head = prefix_output + head_offset;
509 const float *suffix_head = suffix_output + head_offset;
510 float *output_head = output + head_offset;
511
512 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
513 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
514 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
515
516 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
517 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
518 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
519
520 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
521 const float max_lse = fmaxf(p_lse, s_lse);
522 p_lse -= max_lse;
523 s_lse -= max_lse;
524
525 // Step 2-3: 指数还原 → 混合比例 alpha, beta
526 const float p_se = expf(p_lse);
527 const float s_se = expf(s_lse);
528 const float out_se = p_se + s_se;
529 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
530 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
531
532 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
533 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
534 if (pack_offset < head_size) {
535     pack_t p_pack =
536         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
537     pack_t s_pack =
538         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
539     pack_t o_pack;
540
541 #pragma unroll
542     for (int i = 0; i < PACK_SIZE; ++i) {
543         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
544         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
545         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
546         reinterpret_cast<float *>(&o_pack)[i] = o_v;
547     }
548
549     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
550 }
551 }
552
553 // =====
554 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
555 // =====
556 // 面试要点:
557 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
558 //     online (1-pass, 增量更新)
559 //   - Online Softmax 是 FlashAttention 的数学基础
560 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
561 //     + variance)
562 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
563
564 // =====
565 // Phase 3a: Softmax — 三级递进 (面试核心考点)
566 // =====

```

```

567 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
568 // 回答线索: naive(溢出) → safe → online
569
570 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
571 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
572 // 问题: x 值很大时 exp(x) 溢出为 inf
573 template <const int NUM_THREADS = 256>
574 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
575 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
576 // source: LeetCUDA/kernels/softmax/softmax.cu
577 __global__ void softmax_per_token(float *x, float *y, int N) {
578     const int tid = threadIdx.x;
579     const int idx = blockIdx.x * blockDim.x + tid;
580
581     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
582     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
583     if (idx < N)
584         y[idx] = exp_val / exp_sum;
585 }
586
587 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
588 // 面试重点: 为什么 Softmax 需要 Safe?
589 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
590 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
591 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
592 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
593 // Grid: (S, 1, 1)
594 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
595 // source: LeetCUDA/kernels/softmax/softmax.cu
596 template <const int NUM_THREADS = 256>
597 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
598     const int tid = threadIdx.x;
599     const int idx = blockIdx.x * blockDim.x + tid;
600
601     // Pass 1: block reduce max — 找最大值
602     float val = (idx < N) ? x[idx] : -FLT_MAX;
603     float max_val = block_reduce_max<NUM_THREADS>(val);
604
605     // Pass 2: exp(x - max) → block reduce sum
606     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
607     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
608
609     if (idx < N)
610         y[idx] = exp_val / exp_sum;
611 }
612
613 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
614 // 面试重点:
615 // 两种使用场景 (公式不同, 但结果等价):
616 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
617 //       m_new = max(m_old, x_i)
618 //       d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
619 //   2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
620 //       m = max(m1, m2) safe softmax用的max值
621 //       d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
622 //       当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
623 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
624 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
625 //
626 // 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)),
627 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
628 //   (m1,d1): max 和 Σexp(x_j - m1), 覆盖集合 S1
629 //   (m2,d2): max 和 Σexp(x_k - m2), 覆盖集合 S2

```

```

630 //
631 // 合并公式 (对称, 不依赖"新/旧"概念):
632 //   m = max(m1, m2)
633 //   d = d1*exp(m1-m) + d2*exp(m2-m)   ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
634 //
635 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
636 struct __align__(8) MD {
637     float m; // running max
638     float d; // running denominator (sum of exp(x - max))
639 };
640
641 template <const int kWarpSize = WARP_SIZE>
642 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
643     #pragma unroll
644     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
645         MD md2;
646         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpSize);
647         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpSize);
648
649         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
650         MD s_m = (md1.m > md2.m) ? md2 : md1;
651
652         // b_m.d 无需 rescale (其基准 max 已是全局 max),
653         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
654         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
655         md1.m = b_m.m;
656     }
657     return md1;
658 }
659
660 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
661 // Grid: (S, 1, 1)
662 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
663 // source: LeetCUDA/kernels/softmax/softmax.cu
664 template <const int NUM_THREADS = 256>
665 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
666     int tid = threadIdx.x;
667     int idx = blockIdx.x * NUM_THREADS + threadIdx.x;
668     const int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
669     __shared__ MD shared[NUM_WARPS];
670
671     float val = (idx < N) ? x[idx] : -FLT_MAX;
672     int warp = tid / WARP_SIZE;
673     int lane = tid % WARP_SIZE;
674
675     // 初始化: 每个线程持有一个 (max, denom) 对
676     MD md;
677     md.m = idx < N ? val : -FLT_MAX;
678     md.d = idx < N ? 1.0f : 0.0f;
679
680     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d
681     md = warp_reduce_md<WARP_SIZE>(md);
682
683     if (lane == 0)
684         shared[warp] = md;
685     __syncthreads();
686
687     // 第二级归约: 每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
688     md = lane < NUM_WARPS ? shared[lane] : MD{-FLT_MAX, 0.0f};
689     md = warp_reduce_md<WARP_SIZE>(md); // 用WARP_SIZE确保每个lane都能拿到最终结果
690
691     // 用全局 max 和 denom 做最终 softmax
692     float d_inv = __fdividef(1.0f, md.d);

```

```

693 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
694 if (idx < N) {
695     y[idx] = __expf(val - md.m) * d_inv;
696 }
697 }
698
699 // =====
700 // Phase 3b: RMS Normalization (1-pass reduce)
701 // =====
702 // 面试要点:
703 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
704 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
705 //   - Llama 系列使用 RMS Norm
706 //   - grid(N, K/K), block(K): 一行一个 block
707 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
708 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
709 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
710 template <const int NUM_THREADS = 128>
711 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
712     int tid = threadIdx.x;
713     int idx = blockIdx.x * blockDim.x + threadIdx.x;
714     const float epsilon = 1e-5f;
715
716     __shared__ float s_variance;
717     float value = (idx < N * K) ? x[idx] : 0.0f;
718     float variance = value * value;
719     variance = block_reduce_sum<NUM_THREADS>(variance);
720     if (tid == 0)
721         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
722     __syncthreads();
723     if (idx < N * K)
724         y[idx] = (value * s_variance) * g;
725 }
726
727 // RMS Norm + float4
728 // Grid: (N, 1, 1)
729 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
730 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
731 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
732 template <const int NUM_THREADS = 128 / 4>
733 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
734     int tid = threadIdx.x;
735     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
736     const float epsilon = 1e-5f;
737
738     __shared__ float s_variance;
739     float4 reg_x = FLOAT4(x[idx]);
740     float4 variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
741                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
742                                     : 0.0f;
743     variance = block_reduce_sum<NUM_THREADS>(variance);
744     if (tid == 0)
745         s_variance = rsqrtf(variance / (float)K + epsilon);
746     __syncthreads();
747     float4 reg_y;
748     reg_y.x = reg_x.x * s_variance * g;
749     reg_y.y = reg_x.y * s_variance * g;
750     reg_y.z = reg_x.z * s_variance * g;
751     reg_y.w = reg_x.w * s_variance * g;
752     if (idx < N * K)
753         FLOAT4(y[idx]) = reg_y;
754 }
755

```

```

756 // =====
757 // Phase 3c: Layer Normalization (2-pass reduce)
758 // =====
759 // 面试要点:
760 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
761 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
762 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
763 // Grid: (N, 1, 1), 一行一个 block
764 // Block: (128, 1, 1), NUM_THREADS=K=128
765 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
766 template <const int NUM_THREADS = 128>
767 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
768     int tid = threadIdx.x;
769     int idx = blockIdx.x * blockDim.x + threadIdx.x;
770     const float epsilon = 1e-5f;
771
772     __shared__ float s_mean;
773     __shared__ float s_variance;
774     float value = (idx < N * K) ? x[idx] : 0.0f;
775
776     // Pass 1: compute mean
777     float sum = block_reduce_sum<NUM_THREADS>(value);
778     if (tid == 0)
779         s_mean = sum / (float)K;
780     __syncthreads(); // 必须等待 s_mean 对所有线程可见
781
782     // Pass 2: compute variance = (x - mean)2
783     float variance = (value - s_mean) * (value - s_mean);
784     variance = block_reduce_sum<NUM_THREADS>(variance);
785     if (tid == 0)
786         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
787     __syncthreads(); // 必须等待 s_variance 对所有线程可见
788
789     if (idx < N * K)
790         y[idx] = ((value - s_mean) * s_variance) * g + b;
791 }
792
793 // Layer Norm + float4
794 // Grid: (N, 1, 1)
795 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
796 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
797 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
798 template <const int NUM_THREADS = 128 / 4>
799 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
800     int tid = threadIdx.x;
801     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
802     const float epsilon = 1e-5f;
803
804     __shared__ float s_mean;
805     __shared__ float s_variance;
806     float4 reg_x = FLOAT4(x[idx]);
807     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
808
809     float sum = block_reduce_sum<NUM_THREADS>(value);
810     if (tid == 0)
811         s_mean = sum / (float)K;
812     __syncthreads();
813
814     float4 reg_x_hat;
815     reg_x_hat.x = reg_x.x - s_mean;
816     reg_x_hat.y = reg_x.y - s_mean;
817     reg_x_hat.z = reg_x.z - s_mean;
818     reg_x_hat.w = reg_x.w - s_mean;

```

```

819 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
820         reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
821 variance = block_reduce_sum<NUM_THREADS>(variance);
822 if (tid == 0)
823     s_variance = rsqrtf(variance / (float)K + epsilon);
824 __syncthreads();
825
826 float4 reg_y;
827 reg_y.x = reg_x_hat.x * s_variance * g + b;
828 reg_y.y = reg_x_hat.y * s_variance * g + b;
829 reg_y.z = reg_x_hat.z * s_variance * g + b;
830 reg_y.w = reg_x_hat.w * s_variance * g + b;
831 if (idx < N * K)
832     FLOAT4(y[idx]) = reg_y;
833 }
834
835 // =====
836 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
837 // =====
838 // 面试要点:
839 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
840 //     [x1']   [cos(θ)  -sin(θ)] [x1]
841 //     [x2'] = [sin(θ)   cos(θ)] [x2]
842 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
843 //   - token_pos = idx / N: token 在序列中的位置
844 //   - token_idx = idx % N: token 内的维度索引
845 //   - 输入 [seq_len, hidden_size], 输出同形状
846
847 // Grid: ((seq_len * N + 255) / 256, 1, 1)
848 // Block: (256, 1, 1)
849 // source: LeetCUDA/kernels/rope/rope.cu
850 __global__ void rope(float *x, float *out, int seq_len, int N) {
851     int idx = blockIdx.x * blockDim.x + threadIdx.x;
852     float x1 = x[idx * 2];
853     float x2 = x[idx * 2 + 1];
854     int token_pos = idx / N; // 序列位置
855     int token_idx = idx % N; // 维度索引
856
857     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
858     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
859     float sin_v = sinf(token_pos * exp_v);
860     float cos_v = cosf(token_pos * exp_v);
861
862     // 2D 旋转
863     float out1 = x1 * cos_v - x2 * sin_v;
864     float out2 = x1 * sin_v + x2 * cos_v;
865     out[idx * 2] = out1;
866     out[idx * 2 + 1] = out2;
867 }
868
869 // =====
870 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
871 // =====
872 // 面试要点 (Bank Conflict 专题):
873 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
874 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
875 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
876 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
877 //
878 // 转置的四步演进:
879 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
880 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
881 //   BCF:   smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict

```



```

882 // merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
883 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
884
885 // 每个线程处理 1 个元素, block(16,16)
886 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
887 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
888 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
889 // Block: (16, 16, 1)
890 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
891 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
892     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
893     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
894     if (r < row && c < col) {
895         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
896         y[c * row + r] = x[r * col + c];
897     }
898 }
899
900 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
901 // Block: (16, 16, 1)
902 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
903 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
904 __global__ void mat_transpose_padded(
905     float *x, float *y, const int row, const int col) {
906     const int tx = threadIdx.x, ty = threadIdx.y;
907
908     constexpr int TILE = 16;
909     constexpr int PAD = 1;
910     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
911     __shared__ float tile[TILE * 4][TILE + PAD];
912
913     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
914     const int x_c = blockIdx.x * TILE + tx;
915     const int x_r = blockIdx.y * TILE + ty;
916
917     if (x_r * 4 < row && x_c < col) {
918         // Step 1: 4 rows × 1 col → smem (row-major);
919 #pragma unroll
920         for (int i = 0; i < 4; ++i) {
921             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
922         }
923         __syncthreads();
924
925         // Step 2: Transposed read from smem
926         float4 reg_trans;
927         reg_trans.x = tile[tx * 4 + 0][ty];
928         reg_trans.y = tile[tx * 4 + 1][ty];
929         reg_trans.z = tile[tx * 4 + 2][ty];
930         reg_trans.w = tile[tx * 4 + 3][ty];
931
932         // y 空间坐标: y_r = x 的列, y_c = x 的行
933         const int y_r = blockIdx.x * TILE + ty; // = x col
934         const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
935
936         // Coalesced write: y[y_r][y_c .. y_c+3]
937         FLOAT4(y[y_r * row + y_c]) = reg_trans;
938     }
939 }
940
941 // =====
942 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
943 // =====
944 // 面试要点:

```



```

945 // - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
946 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
947 // - 不同 K 值对应不同分块策略:
948 //   K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
949 //   K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
950 //   K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
951 //
952 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
953
954 // ---- SGEMV K32: 基础 warp-per-row ----
955 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
956 // grid(M/4), 每个 warp 负责一行
957 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
958 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
959 // Block: (32, 4, 1), 每行 1 warp
960 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
961 // source: LeetCUDA/kernels/sgemv/sgemv.cu
962 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
963     int tx = threadIdx.x; // 0~31
964     int ty = threadIdx.y; // 0~3
965     int lane = tx % WARP_SIZE; // 0~31
966     int m = blockIdx.x * blockDim.y + ty; // 全局行号
967     if (m < M) {
968         float sum = 0.0f;
969         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
970         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
971         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
972 #pragma unroll
973         for (int w = 0; w < NUM_ITERS; ++w) {
974             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
975             int k = w * WARP_SIZE + lane;
976             sum += a[m * K + k] * x[k];
977         }
978         sum = warp_reduce_sum<WARP_SIZE>(sum);
979         // 每个 warp 处理一行, lane 0 写回结果
980         if (lane == 0)
981             y[m] = sum;
982     }
983 }
984
985 // ---- SGEMV K128: float4 向量化 ----
986 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
987 // Grid: ((M + 3) / 4, 1, 1)
988 // Block: (32, 4, 1)
989 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
990 // source: LeetCUDA/kernels/sgemv/sgemv.cu
991 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
992     int tx = threadIdx.x; // 0~31
993     int ty = threadIdx.y; // 0~3
994     int lane = tx % WARP_SIZE; // 0~31
995     int m = blockDim.y * blockIdx.x + ty;
996
997     if (m < M) {
998         float sum = 0.0f;
999         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1000         const int NUM_ITERS = ((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
1001 #pragma unroll
1002         for (int w = 0; w < NUM_ITERS; ++w) {
1003             int k = (w * WARP_SIZE + lane) * 4;
1004             float4 reg_x = FLOAT4(x[k]);
1005             float4 reg_a = FLOAT4(a[m * K + k]);
1006             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1007                    reg_a.w * reg_x.w);

```

```

1008     }
1009     sum = warp_reduce_sum<WARP_SIZE>(sum);
1010     if (lane == 0)
1011         y[m] = sum;
1012 }
1013 }
1014
1015 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
1016 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1017 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1018 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1019 // Block: (32, 4, 1)
1020 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
1021 // source: LeetCode/kernels/sgemv/sgemv.cu
1022 template <const int ROW_PER_WARP = 2>
1023 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1024     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
1025     int tx = threadIdx.x;
1026     int ty = threadIdx.y;
1027     int lane = tx % WARP_SIZE;
1028     int k = lane % K_WARP_SIZE; // 0~15
1029     int m = (blockDim.y * blockIdx.x + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
1030     if (m < M) {
1031         float sum = A[m * K + k] * x[k];
1032         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1033         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
1034         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1035         if (k == 0)
1036             y[m] = sum;
1037     }
1038 }
1039
1040 // =====
1041 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1042 // =====
1043 // 面试要点 (GEMM 优化五层金字塔):
1044 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1045 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1046 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1047 //   load/store 指令数 Level 4 — Tensor Core (MMA
1048 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1049 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1050 //
1051 // 计算密度递进:
1052 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1053 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1054 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1055 //
1056 // =====
1057 // Phase 7a: SGEMM (非 Tensor Core 路径)
1058 // =====
1059
1060 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
1061 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1062 // C = A × B, C[M, N] = A[M, K] × B[K, N]
1063 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1064 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1065 // Block: (32, 32, 1), 1024 线程
1066 // source: LeetCode/kernels/sgemm/sgemm.cu
1067 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1068     constexpr int BM = 32; // vec 版: 32×4 = 128
1069     constexpr int BN = 32; // vec 版: 32×4 = 128
1070     constexpr int BK = 32;

```

```

1071 __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1072
1073 int bx = blockIdx.x;
1074 int by = blockIdx.y;
1075 int tx = threadIdx.x;
1076 int tid = threadIdx.y * blockDim.x + tx;
1077
1078 // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1079 int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1080 int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1081 int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1082 int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1083 int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1084 int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1085
1086 float sum = 0.f; // 遍历完整的K, slice K;
1087 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1088     int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1089     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1090     s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1091     int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1092     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1093     s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1094     __syncthreads(); // 确保整个 smem tile 加载完毕
1095
1096 #pragma unroll
1097     for (int k = 0; k < BK; ++k) {
1098         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1099         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1100         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1101     }
1102     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1103 }
1104 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1105 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1106 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1107 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1108 }
1109
1110 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1111 // 在 Level 1 基础上引入两层优化:
1112 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1113 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1114 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1115 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1116 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1117 // 1024 x 16 = 16384 = 128 x 128 ✓
1118 //
1119 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1120 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1121 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1122 //
1123 // 加载映射 (每线程 4 个元素, float4):
1124 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1125 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1126 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1127 //
1128 // △ Bank Conflict 提示 (面试加分点):
1129 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1130 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1131 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1132 //
1133 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)

```

```

1134 // Block: (32, 32, 1), 1024 线程
1135 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1136 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1137 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1138     constexpr int BM = 128;
1139     constexpr int BN = 128;
1140     constexpr int BK = 32;
1141     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1142     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1143
1144     int bx = blockIdx.x;
1145     int by = blockIdx.y;
1146     int tx = threadIdx.x;
1147     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1148
1149     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1150     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1151     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1152     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1153     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1154     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1155
1156     int load_gmem_a_m = by * BM + load_smem_a_m;
1157     int load_gmem_b_n = bx * BN + load_smem_b_n;
1158
1159     // 4x4 Thread Tile 基址 (独立于加载映射), 这里compute索引的计算逻辑要和
1160     // load索引的计算逻辑分开, load/compute是可以独立索引的, 理解这点很重要。
1161     // 目标C Tile为[BM,BN]=[128x128], 有32x32线程, 则每个线程处理4x4 tile
1162     // 那么, 就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
1163     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1164     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1165
1166     float sum[4][4] = {0.f};
1167     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1168         int load_gmem_a_k = bk * BK + load_smem_a_k;
1169         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1170         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
1171         int load_gmem_b_k = bk * BK + load_smem_b_k;
1172         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1173         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
1174         __syncthreads();
1175
1176     #pragma unroll
1177     for (int k = 0; k < BK; ++k) {
1178         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1179         float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1180                             s_a[comp_smem_a_m_base + 1][k],
1181                             s_a[comp_smem_a_m_base + 2][k],
1182                             s_a[comp_smem_a_m_base + 3][k]};
1183         float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1184                             s_b[k][comp_smem_b_n_base + 1],
1185                             s_b[k][comp_smem_b_n_base + 2],
1186                             s_b[k][comp_smem_b_n_base + 3]};
1187
1188     #pragma unroll
1189     for (int i = 0; i < 4; ++i) {
1190     #pragma unroll
1191         for (int j = 0; j < 4; ++j) {
1192             sum[i][j] += a_vals[i] * b_vals[j];
1193         }
1194     }
1195     __syncthreads();
1196 }

```

```

1197 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1198 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1199 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1200 #pragma unroll
1201 for (int i = 0; i < 4; ++i) {
1202     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1203     float4 reg_c;
1204     reg_c.x = sum[i][0];
1205     reg_c.y = sum[i][1];
1206     reg_c.z = sum[i][2];
1207     reg_c.w = sum[i][3];
1208     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1209 }
1210 }
1211 }
1212
1213 // =====
1214 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1215 // =====
1216 // 面试要点:
1217 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1218 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1219 // - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1220 // 寄存器)
1221 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1222 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1223 //
1224 // ★ TN 布局详解 (面试高频考点):
1225 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1226 // BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1227 // N = Normal (列优先, BLAS 原生格式)
1228 // T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1229 // 第一个字母 → A 的 op, 第二个字母 → B 的 op
1230 // 所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1231 // BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1232 // 视角下: TN = A row-major [M×K], B^T row-major [N×K] (等价于 B col-major [K×N]) 在 cuBLAS
1233 // 调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1234 // T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1235 // N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1236 //
1237 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1238 // T = row-major (行优先, 对 BLAS 来说是 transposed)
1239 // N = col-major (列优先, BLAS native = normal)
1240 //
1241 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[M×N] = A[M×K] × B[K×N]
1242 // - A: row-major [M, K], B: row-major [K, N]
1243 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1244 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1245 //
1246 // TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 B^T=[N×K] row-major 存储 (A 行优先, B 列优先)
1247 // - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1248 // - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1249 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1250 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1251 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1252 //
1253 // WGMMMA (Warp Group MMA, Hopper+):
1254 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1255 // - m64n128k16: 一次处理 64x128x16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1256 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1257 // threads) 做计算
1258 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1259

```

```

1260 // =====
1261 // Phase 7b-1: MMA PTX 宏定义
1262 // =====
1263
1264 // ---- gmem → smem: cp.async ----
1265 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1266 //   - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1267 //   - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1268 //     N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1269 //   - wait_all: 等价于 commit_group + wait_group 0。
1270 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1271 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1272 #define CP_ASYNC_WAIT_GROUP(n) \
1273     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1274
1275 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1276 #define CP_ASYNC_CG(dst, src, bytes) \
1277     asm volatile( \
1278         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1279         "l"(src), "n"(bytes))
1280
1281 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1282 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1283 // 每次加载 8x8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1284 // aligned: 要求 128-bit 对齐, trans: 转置加载
1285 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1286     asm volatile( \
1287         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1288         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1289         : "r"(addr))
1290
1291 #define LDMATRIX_X2(R0, R1, addr) \
1292     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1293         : "=r"(R0), "=r"(R1) \
1294         : "r"(addr))
1295
1296 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1297 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1298 #define LDMATRIX_X2_T(R0, R1, addr) \
1299     asm volatile( \
1300         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1301         : "=r"(R0), "=r"(R1) \
1302         : "r"(addr))
1303
1304 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1305 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1306 // row.col: A 是 row-major, B 是 col-major
1307 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1308 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1309 // C 矩阵大小 16x8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1310 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1311     asm volatile( \
1312         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1313         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1314         : "=r"(RD0), "=r"(RD1) \
1315         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1316         "r"(RC1))
1317
1318 // ---- MMA 辅助函数 ----
1319 // div_ceil: 整数除法向上取整
1320 #define HOST_DEVICE_INLINE __device__ __host__ inline
1321 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1322     return (a % b != 0) ? (a / b + 1) : (a / b);

```



```

1323 }
1324
1325 // =====
1326 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1327 // =====
1328 // 面试重点 — Tile Hierarchy:
1329 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1330 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1331 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1332 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1333 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1334 //
1335 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1336 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1337 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1338 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1339 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1340 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1341 //
1342 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1343 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1344 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1345 //   - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1346 //   - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1347 //
1348 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1349 // Block: (256, 1, 1), 8 warps
1350 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1351 // =====
1352 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1353           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1354           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1355           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1356 __global__ void __launch_bounds__(256)
1357   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1358   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1359   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1360   const int by = blockIdx.y;
1361   constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1362   constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1363   constexpr int BK = MMA_K; // 16
1364
1365   // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1366   // TN 布局: s_a[BM][BK]=[128][16] (A row-major), s_b[BN][BK]=[128][16] (B^T
1367   // row-major, 即 B col-major [KxN] 在 smem 中按 B^T[NxK] 存储)
1368   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1369   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1370   extern __shared__ half smem[];
1371   half *s_a = smem;
1372   half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1373   constexpr int s_a_stage_offset = BM * BK; // 128*16
1374   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)xBK(16) = B^T row-major
1375   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1376   const int warp_id = tid / WARP_SIZE; // 0~7
1377   const int lane_id = tid % WARP_SIZE; // 0~31
1378   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1379   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1380
1381   // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1382   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1383   int load_smem_a_m = tid / 2; // 0~127
1384   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1385   int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)

```

```

1386 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1387 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1388 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1389 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1390     return;
1391
1392 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1393 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1394 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。
1395 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1396 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1397
1398 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1399 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1400 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1401
1402 // 预加载前 (K_STAGE-1) 个 stage
1403 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1404 #pragma unroll
1405 for (int k = 0; k < (K_STAGE - 1); ++k) {
1406     int load_gmem_a_k = k * BK + load_smem_a_k;
1407     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1408     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1409         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1410     );
1411     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1412
1413     int load_gmem_b_k = k * BK + load_smem_b_k;
1414     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1415     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1416     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1417         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1418     );
1419     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1420
1421     CP_ASYNC_COMMIT_GROUP();
1422 }
1423
1424 CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1425 __syncthreads();
1426
1427 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1428 const int NUM_K_TILES = div_ceil(K, BK);
1429 #pragma unroll
1430 for (int k = 0; k < NUM_K_TILES; ++k) {
1431     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1432     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1433
1434     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
1435     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k (row-major, 内维连续的是 K)
1436     if (k + K_STAGE - 1 < NUM_K_TILES) {
1437         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1438         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1439         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1440         // B^T: row-major [n][k], 内维连续的是 K △
1441         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1442
1443         uint32_t load_smem_a_ptr =
1444             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1445                 load_smem_a_m * BK + load_smem_a_k) *
1446                 sizeof(half));
1447         CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1448     }
1449 }

```



```

1449     uint32_t load_smem_b_ptr =
1450         (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1451             load_smem_b_n * BK + load_smem_b_k) *
1452             sizeof(half));
1453     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1454     CP_ASYNC.COMMIT_GROUP();
1455 }
1456
1457 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1458 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1459 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1460 uint32_t RA[VAL_TILE_M][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1461 uint32_t RB[VAL_TILE_N][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1462
1463 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1464 #pragma unroll
1465 for (int i = 0; i < VAL_TILE_M; ++i) {
1466     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1467     int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1468     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1469     int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1470     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1471     uint32_t lane_smem_a_ptr =
1472         (smem_a_base_ptr +
1473             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1474             sizeof(half));
1475     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1476 }
1477
1478 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1479 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1480 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1481 #pragma unroll
1482 for (int j = 0; j < VAL_TILE_N; ++j) {
1483     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1484     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1485     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1486     int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1487     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1488     uint32_t lane_smem_b_ptr =
1489         (smem_b_base_ptr +
1490             (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1491             sizeof(half));
1492     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1493 }
1494
1495 // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1496 #pragma unroll
1497 for (int i = 0; i < VAL_TILE_M; ++i) {
1498     #pragma unroll
1499     for (int j = 0; j < VAL_TILE_N; ++j) {
1500         HMMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1501             RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1502             RB[j][0], RB[j][1], // B fragment
1503             RC[i][j][0], RC[i][j][1]);
1504     }
1505 }
1506
1507 // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1508 if (k + K_STAGE - 1 < NUM_K_TILES) {
1509     CP_ASYNC.WAIT_GROUP(K_STAGE - 2);
1510 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1511     CP_ASYNC.WAIT_GROUP(0);

```

```

1512 }
1513 __syncthreads();
1514 }
1515
1516 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1517 {
1518     for (int i = 0; i < VAL_TILE_M; ++i) {
1519         uint32_t RC0[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1520         uint32_t RC1[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1521         // =====
1522         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1523         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1524         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1525         //
1526         //      cols: c0-1    c2-3    c4-5    c6-7
1527         //  r0:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1528         //  r1:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1529         //  r2:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
1530         //  ...                               |
1531         //  r7:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1532         //  r8:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1533         //  r9:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1534         // r10:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
1535         //  ...                               |
1536         // r15: T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1537         //
1538         // Within a 4-lane group (e.g. T0-T3 for row 0):
1539         //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1540         //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1541         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1542         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1543         // =====
1544     #pragma unroll
1545     for (int j = 0; j < VAL_TILE_N; ++j) {
1546         RC0[j][0] = RC[i][j][0];
1547         RC1[j][0] = RC[i][j][1];
1548         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1549         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1550         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1551         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1552         RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1553         RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1554     }
1555     // 每 4 个 lane 中只有 lane 0 做 128-bit store
1556     // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行) :
1557     //
1558     // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1559     // |-----|-----|-----|-----|
1560     // | 0~3   | 0       | row 0     | row 8     |
1561     // | 4~7   | 1       | row 1     | row 9     |
1562     // | 8~11  | 2       | row 2     | row 10    |
1563     // | 12~15 | 3       | row 3     | row 11    |
1564     // | 16~19 | 4       | row 4     | row 12    |
1565     // | 20~23 | 5       | row 5     | row 13    |
1566     // | 24~27 | 6       | row 6     | row 14    |
1567     // | 28~31 | 7       | row 7     | row 15    |
1568     //
1569     if (lane_id % 4 == 0) {
1570         // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1571         int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1572         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1573     #pragma unroll
1574     for (int j = 0; j < VAL_TILE_N; ++j) {

```

```

1575 // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1576 int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1577 int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1578 int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1579 int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1580 // 128-bit store: 一次写入 8 个 half
1581 *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1582     *reinterpret_cast<float4*>(&RC0[j][0]);
1583 *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1584     *reinterpret_cast<float4*>(&RC1[j][0]);
1585 }
1586 }
1587 }
1588 }
1589 }
1590
1591 // =====
1592 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1593 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1594 // =====
1595 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除) :
1596 // - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1597 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way) 。
1598 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1599 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:
1600 //   rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1601 //   rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1602 //   rows 8~11: repeat rows 0~3 pattern
1603 //   rows 12~15: repeat rows 4~7 pattern
1604 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free) 。
1605 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1606 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1607 //   对 MMA m16n8k16 的 BK=16 而言刚好满足。
1608 //
1609 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1610 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1611
1612 // ---- Swizzle 辅助函数 ----
1613
1614 // swizzle_permuted_j: 对 smem 列索引 j 做 XOR 置换, 消除 bank conflict。
1615 // i: row index; j: col index.
1616 // e.g kColStride = 16, kStep = 8 -> load 8 half as 128 bits memory issue.
1617 // 公式: ((j / kStep) ^ (i / 4)) % (kColStride / kStep) * kStep
1618 // 用位运算展开 (kStep=8) : (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3
1619 // 限制: kColStride ≤ 16 (BK ≤ 16), kStep ∈ {4, 8}, kColStride % kStep == 0
1620 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1621 template <const int kColStride = 16, const int kStep = 8>
1622 static __device__ __forceinline__ int swizzle_permuted_j(int i, int j) {
1623     // for col_stride > 16, we have to permute it using col major ZigZag order.
1624     // e.g, A smem logical layout [Br,d]=[Br,64] -> store layout [4][Br][16].
1625     static_assert(kColStride <= 16, "kColStride must <= 16");
1626     // swizzle: ((int(j / kStep) ^ int(i / 4)) % int(kColStride / kStep)) * kStep;
1627     static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1628     static_assert(kColStride % kStep == 0,
1629         "kColStride must be multiple of kStep.");
1630     if constexpr (kStep == 8) {
1631         return (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3;
1632     } else {
1633         static_assert(kStep == 4);
1634         return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
1635     }
1636 }
1637

```

```

1638 // swizzle_permuted_A_j: A 矩阵专用封装 (kMmaAtomK=16, kStep=8) 。
1639 // 16 行 (一个 MMA atom 的 M 维) 内的 swizzle pattern:
1640 // -----
1641 // -col 0~16, step 8--
1642 // -----
1643 // | row 0 | (0, 8) |
1644 // | row 1 | (0, 8) |
1645 // | row 2 | (0, 8) |
1646 // | row 3 | (0, 8) |
1647 // -----
1648 // | row 4 | (8, 0) |
1649 // | row 5 | (8, 0) |
1650 // | row 6 | (8, 0) |
1651 // | row 7 | (8, 0) |
1652 // -----
1653 // | row 8 | (0, 8) |
1654 // | row 9 | (0, 8) |
1655 // | row 10 | (0, 8) |
1656 // | row 11 | (0, 8) |
1657 // -----
1658 // | row 12 | (8, 0) |
1659 // | row 13 | (8, 0) |
1660 // | row 14 | (8, 0) |
1661 // | row 15 | (8, 0) |
1662 // -----
1663 // source: LeetCUDA/kernels/hgemmm/mma/swizzle/hgemmm_mma_stage_tn_swizzle.cu
1664 template <const int kMmaAtomK = 16>
1665 static __device__ __forceinline__ int swizzle_permuted_A_j(int i, int j) {
1666     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1667 }
1668
1669 // swizzle_permuted_B_j: B 矩阵专用封装 (与 A 相同的 pattern) 。
1670 // B^T smem 布局 [BN][BK]=[128][16] 在 BK 维做 swizzle,
1671 // pattern 与 A 完全相同 (kMmaAtomK=16, kStep=8) 。
1672 // source: LeetCUDA/kernels/hgemmm/mma/swizzle/hgemmm_mma_stage_tn_swizzle.cu
1673 template <const int kMmaAtomK = 16>
1674 static __device__ __forceinline__ int swizzle_permuted_B_j(int i, int j) {
1675     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1676 }
1677
1678 // =====
1679 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1680 // =====
1681 // 与 Phase 7b-2 (hgemmm_mma_stages_tn) 的核心区别:
1682 // - 所有 smem 地址计算加 swizzle_permuted_A_j/swizzle_permuted_B_j, 消除 bank conflict
1683 // - VAL_TILE_M/N → VAL_TILE_M/N (与源 kernel 命名一致)
1684 // - 其余 tile hierarchy、pipeline、epilogue 与统一循环版完全一致
1685 //
1686 // Tile Hierarchy:
1687 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1688 //   MMA Tile (more warps): 2×4=8 个 MMA atom → [2×16, 8×4]=[32,32]
1689 //   WARP Tile (more values): 4×4=16 expand → [32×4, 32×4]=[128,128]
1690 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1691 //         → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
1692 //
1693 // TN 布局: C[M×N] = A[M×K] × B^T[N×K]
1694 // - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1695 // - B^T[N][K]: row-major → 全局索引 B[n*K+k]
1696 // - ldmatrix A: x4 (非转置), A row-major 原生匹配
1697 // - ldmatrix B: x2 (非转置), B^T row-major 逐行加载 = B 的列
1698 // - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1699 //
1700 // 统一循环设计 (同 hgemmm_mma_stages_tn) :

```

```

1701 // - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1702 // - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1703 // - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1704 // - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1705 //
1706 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1707 // Block: (256, 1, 1), 8 warps
1708 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1709 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1710          const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1711          const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1712          const int K_STAGE = 3,
1713          const bool BLOCK_SWIZZLE = false>
1714 __global__ void __launch_bounds__(256)
1715   hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1716   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1717   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1718   const int by = blockIdx.y;
1719   constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1720   constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1721   constexpr int BK = MMA_K; // 16
1722
1723   // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B (与 hgemm_mma_stages_tn 相同)
1724   extern __shared__ half smem[];
1725   half *s_a = smem;
1726   half *s_b = smem + K_STAGE * BM * BK;
1727   constexpr int s_a_stage_offset = BM * BK; // 128*16
1728   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ B^T row-major
1729   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1730   const int warp_id = tid / WARP_SIZE; // 0~7
1731   const int lane_id = tid % WARP_SIZE; // 0~31
1732   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1733   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1734
1735   // 线程到 global memory 的映射 (用于加载 A 和 B)
1736   int load_smem_a_m = tid / 2; // 0~127
1737   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1738   int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1739   int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1740   int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号
1741   int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号
1742   if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1743     return;
1744
1745   // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
1746   // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps. 为了能处理更大的C tile,
1747   // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是Value Tile的作用.
1748   // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念.
1749   uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1750
1751   // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1752   uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1753   uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1754
1755   // 预加载前 (K_STAGE-1) 个 stage
1756   // * swizzle 区别: cp.async 的目标 smem 地址加 swizzle_permuted_A_j/swizzle_permuted_B_j
1757   #pragma unroll
1758   for (int k = 0; k < (K_STAGE - 1); ++k) {
1759     int load_gmem_a_k = k * BK + load_smem_a_k;
1760     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1761     int load_gmem_b_k = k * BK + load_smem_b_k;
1762     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1763

```

```

1764     uint32_t load_smem_a_ptr =
1765         (smem_a_base_ptr +
1766          (k * s_a_stage_offset + load_smem_a_m * BK +
1767           swizzle_permuted_A_j < MMA_K > (load_smem_a_m, load_smem_a_k)) *
1768           sizeof(half));
1769     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1770
1771     uint32_t load_smem_b_ptr =
1772         (smem_b_base_ptr +
1773          (k * s_b_stage_offset + load_smem_b_n * BK +
1774           swizzle_permuted_B_j < MMA_K > (load_smem_b_n, load_smem_b_k)) *
1775           sizeof(half));
1776     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1777
1778     CP_ASYNC_COMMIT_GROUP();
1779 }
1780
1781 CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1782 __syncthreads();
1783
1784 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1785 const int NUM_K_TILES = div_ceil(K, BK);
1786 #pragma unroll
1787 for (int k = 0; k < NUM_K_TILES; ++k) {
1788     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1789     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1790
1791     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
1792     // swizzle 区别: smem 地址加 swizzle_permuted_*_j
1793     if (k + K_STAGE - 1 < NUM_K_TILES) {
1794         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1795         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1796         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1797         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1798
1799         uint32_t load_smem_a_ptr =
1800             (smem_a_base_ptr +
1801              (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
1802               swizzle_permuted_A_j < MMA_K > (load_smem_a_m, load_smem_a_k)) *
1803               sizeof(half));
1804         CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1805
1806         uint32_t load_smem_b_ptr =
1807             (smem_b_base_ptr +
1808              (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
1809               swizzle_permuted_B_j < MMA_K > (load_smem_b_n, load_smem_b_k)) *
1810               sizeof(half));
1811         CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1812         CP_ASYNC_COMMIT_GROUP();
1813     }
1814
1815     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1816     // * swizzle 区别: ldmatrix 的源 smem 地址也加 swizzle_permuted_*_j
1817     uint32_t RA[VAL_TILE_M][4];
1818     uint32_t RB[VAL_TILE_N][2];
1819
1820     // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1821     #pragma unroll
1822     for (int i = 0; i < VAL_TILE_M; ++i) {
1823         int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1824         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1825         int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1826         uint32_t lane_smem_a_ptr =

```

```

1827         (smem_a_base_ptr +
1828         (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1829         swizzle_permuted_A_j<MMA_K>(lane_smem_a_m, lane_smem_a_k)) *
1830         sizeof(half));
1831     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1832 }
1833
1834 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1835 #pragma unroll
1836 for (int j = 0; j < VAL_TILE_N; ++j) {
1837     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1838     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1839     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1840     uint32_t lane_smem_b_ptr =
1841         (smem_b_base_ptr +
1842         (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1843         swizzle_permuted_B_j<MMA_K>(lane_smem_b_n, lane_smem_b_k)) *
1844         sizeof(half));
1845     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1846 }
1847
1848 // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1849 #pragma unroll
1850 for (int i = 0; i < VAL_TILE_M; ++i) {
1851     #pragma unroll
1852     for (int j = 0; j < VAL_TILE_N; ++j) {
1853         HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1854                 RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1855                 RB[j][0], RB[j][1], // B fragment
1856                 RC[i][j][0], RC[i][j][1]);
1857     }
1858 }
1859
1860 // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1861 if (k + K_STAGE - 1 < NUM_K_TILES) {
1862     CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1863 } else {
1864     CP_ASYNC_WAIT_GROUP(0);
1865 }
1866 __syncthreads();
1867 }
1868
1869 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1870 // 与 hgemv_mma_stages_tn 完全一致的 epilogue 模式
1871 {
1872     for (int i = 0; i < VAL_TILE_M; ++i) {
1873         uint32_t RC0[VAL_TILE_N][4];
1874         uint32_t RC1[VAL_TILE_N][4];
1875     #pragma unroll
1876     for (int j = 0; j < VAL_TILE_N; ++j) {
1877         RC0[j][0] = RC[i][j][0];
1878         RC1[j][0] = RC[i][j][1];
1879         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1880         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1881         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1882         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1883         RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1884         RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1885     }
1886     if (lane_id % 4 == 0) {
1887         int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1888         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1889     #pragma unroll

```



```

1890     for (int j = 0; j < VAL_TILE_N; ++j) {
1891         int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1892         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1893         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1894         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1895         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1896             *reinterpret_cast<float4*>(&RC0[j][0]);
1897         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1898             *reinterpret_cast<float4*>(&RC1[j][0]);
1899     }
1900 }
1901 }
1902 }
1903 }
1904
1905 // =====
1906 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
1907 // =====
1908 // 面试要点 (CuTe vs 手写 MMA PTX) :
1909 //   - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
1910 //     Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
1911 //   - 与手写 MMA PTX (Phase 7b) 的对比:
1912 //     - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
1913 //     - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
1914 //       cute::copy / cute::gemm 自动展开为高效指令序列
1915 //   - 核心抽象 ("CuTe 五要素") :
1916 //     1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
1917 //     2. TiledMMA: 描述 MMA 指令 + warp/wargroup 排布 + value tile 的 compile-time type
1918 //     3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
1919 //     4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
1920 //     5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
1921 //
1922 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
1923 //   MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
1924 //     → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
1925 //     → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
1926 //     → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
1927 //
1928 // 调参口诀 (面试速记) :
1929 //   BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
1930 //   Swizzle<B,M,S>: B=bank 维度 bit, M=M维 bit, S=stride bit → Swizzle<3,3,3> = 512 values = 1024B
1931 //   kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
1932 //   EURepeat: 增加 warp 数 → 更多并行但每个 warp 做更少 MMA → 寄存器压力降低
1933 //
1934 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
1935 //   - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
1936 //   - CuTe Swizzle: SmemLayout 声明式指定 swizzle pattern, cute::copy 自动应用
1937 //   - CuTe 优势: 类型安全, 编译器可做更激进的优化, 布局变更只需改类型定义
1938 //
1939 // ★ 本实现的 Tile 配置:
1940 //   BM=128, BN=256, BK=32, kStage=2
1941 //   MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1, ValTile=32x32x16
1942 //   Smem Swizzle<3,3,3>: 512-value XOR permutation, 消除 ldmatrix bank conflict
1943 //   Threads: size(MMA{}) = 128 (4 warps × 2x2 EU repeat)
1944 //   Smem: ~49KB (Stage=2, A[128×32] + B[256×32] × half)
1945 //
1946 // 参考: LeetCode/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
1947 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
1948 // =====
1949
1950 #if defined(NOTES_V2_ENABLE_CUTE)
1951
1952 #include <cute/tensor.hpp>

```

```

1953
1954 // =====
1955 // Phase 7d-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
1956 // =====
1957 // TN 布局:  $C[M \times N] = A[M \times K] \times B^T[N \times K]$ 
1958 //   -  $A[M][K]$  row-major,  $B^T[N][K]$  row-major (等价  $B$  col-major  $[K \times N]$ )
1959 //   - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
1960 //
1961 // Pipeline 流程 (每个 stage 对应一次完整的  $G \rightarrow S \rightarrow R \rightarrow MMA \rightarrow R \rightarrow S \rightarrow G$  链条):
1962 //   1. PREFETCH: 预加载  $kStage-1$  个 tile ( $G \rightarrow S$  via cp.async)
1963 //   2. 主循环 over  $K$  tiles:
1964 //       a.  $S \rightarrow R$ : cute::copy(s2r_tiled_copy, sA, rA) ← 自动展开为 ldmatrix
1965 //       b. MMA: cute::gemm(tiled_mma, rD, rA, rB, rD) ← 自动展开为 mma.sync
1966 //       c.  $G \rightarrow S$  (next tile): cute::copy(g2s_tiled_copy, gA, sA) ← cp.async
1967 //       d. Pipeline wait + smem stage advance
1968 //   3. Epilogue:  $R \rightarrow S$  (via UniversalCopy)  $\rightarrow S \rightarrow G$  (128-bit store)
1969 //
1970 // 面试对比 — 手写 MMA vs CuTe 代码量:
1971 //   手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
1972 //   CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
1973 template <typename T, int BM, int BN, int BK, int kStage, typename TiledMMA,
1974           typename G2SCopyA, typename G2SCopyB, typename SmemLayoutA,
1975           typename SmemLayoutB, typename SmemLayoutC, typename S2RCopyAtomA,
1976           typename S2RCopyAtomB, typename R2SCopyAtomC, typename S2GCopyAtomC,
1977           typename S2GCopyC>
1978 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
1979                                           int n, int k) {
1980     using namespace cute;
1981
1982     // 动态 shared memory: A tile + B tile (C epilogue 复用 A tile 的空间)
1983     extern __shared__ T shm_data[];
1984     T *Ashm = shm_data;
1985     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
1986
1987     int idx = threadIdx.x;
1988     int ix = blockIdx.x; // N 方向 block 索引 (本实现默认不开启 BlockSwizzle)
1989     int iy = blockIdx.y; // M 方向 block 索引
1990     if (iy * BM >= m || ix * BN >= n) return;
1991
1992     // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
1993     // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
1994     Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
1995                             make_stride(k, Int<1>{}));
1996     Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
1997                             make_stride(k, Int<1>{}));
1998     Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
1999                             make_stride(n, Int<1>{}));
2000
2001     // local_tile: 从全局 Tensor 切出当前 CTA 负责的 tile
2002     // make_coord(iy, _): M 维固定为 iy, K 维自动按 BK 分 tile (_ = 通配符)
2003     Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2004     Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2005     Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2006
2007     // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2008     auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2009     auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2010
2011     // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2012     TiledMMA tiled_mma;
2013     auto thr_mma = tiled_mma.get_slice(threadIdx.x);
2014     auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2015     auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)

```

```

2016 auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2017 clear(tCrD); // 累加器清零
2018
2019 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (使用 cp.async 128-bit)
2020 G2SCopyA g2s_tiled_copy_a;
2021 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
2022 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, k_tile)
2023 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2024
2025 G2SCopyB g2s_tiled_copy_b;
2026 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
2027 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB);
2028 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2029
2030 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2031 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2032 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2033 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
2034 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2035 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2036
2037 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2038 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
2039 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2040 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2041
2042 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满流水线
2043 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2044 #pragma unroll
2045 for (int istage = 0; istage < kStage - 1; ++istage) {
2046     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2047               tAsA_copy(_, _, _, istage));
2048     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2049               tBsB_copy(_, _, _, istage));
2050     cp_async_fence();
2051     ++itile_to_read;
2052     ++ismem_write;
2053 }
2054 cp_async_wait<kStage - 2>();
2055 __syncthreads();
2056
2057 // 主循环: over K tiles, 每个 K tile 内再分 MMA_K 步迭代
2058 int ntile = k / BK; // K 方向总 tile 数
2059 int ik = 0;
2060 // 首轮 S→R 加载 (在进入循环前先加载第一个 ik slice)
2061 cute::copy(s2r_tiled_copy_a, tAsA(_, _, ik, ismem_read), tCrA_view(_, _, ik));
2062 cute::copy(s2r_tiled_copy_b, tBsB(_, _, ik, ismem_read), tCrB_view(_, _, ik));
2063
2064 #pragma unroll 1
2065 for (int itile = 0; itile < ntile; ++itile) {
2066     int nk = size<2>(tCrA); // 每个 K tile 内的 MMA_K 步数
2067
2068     #pragma unroll
2069     for (int ik = 0; ik < nk; ++ik) {
2070         int ik_next = (ik + 1) % nk;
2071
2072         if (ik == nk - 1) {
2073             // 等待下一批 smem 数据就绪 (pipeline 同步)
2074             cp_async_wait<kStage - 2>();
2075             __syncthreads();
2076             ismem_read = (ismem_read + 1) % kStage;
2077         }
2078

```

```

2079 // S→R: 加载下一组 A/B fragment 到寄存器 (用 ik_next 预取)
2080 cute::copy(s2r_tiled_copy_a, tAsA(_, _, ik_next, ismem_read),
2081            tCrA_view(_, _, ik_next));
2082 cute::copy(s2r_tiled_copy_b, tBsB(_, _, ik_next, ismem_read),
2083            tCrB_view(_, _, ik_next));
2084
2085 if (ik == 0) {
2086     // G→S: 提交下一个 K tile 的异步拷贝 (流水线加载)
2087     if (itile_to_read < ntile) {
2088         cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2089                  tAsA_copy(_, _, _, ismem_write));
2089         cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2090                  tBsB_copy(_, _, _, ismem_write));
2091         ++itile_to_read;
2092         ismem_write = (ismem_write + 1) % kStage;
2093     }
2094     cp_async_fence();
2095 }
2096
2097 // MMA: 发射 Tensor Core 指令 (自动展开为 mma.sync.aligned.m16n8k16)
2098 cute::gemm(tiled_mma, tCrD, tCrA(_, _, ik), tCrB(_, _, ik), tCrD);
2099 }
2100 }
2101
2102 // Epilogue: D寄存器 → shared memory → global memory
2103 // 使用 A tile 的最后一个 stage 作为 C 的 scratchpad (节省 smem)
2104 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2105
2106 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2107 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2108 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2109 auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2110 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2111
2112 // S2G TiledCopy: shared memory → global memory (128-bit store)
2113 S2GCopyC s2g_tiled_copy_c;
2114 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_thread_slice(idx);
2115 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2116 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2117
2118 // group_modes: 将多维 Tensor 的某些 mode 合并, 简化遍历
2119 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2120 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2121
2122 int step = size<3>(tCsC_r2s); // pipeline depth of C smem
2123 #pragma unroll
2124 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2125     // R→S: 寄存器写回 shared memory
2126     #pragma unroll
2127     for (int j = 0; j < step; ++j) {
2128         auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2129         cute::copy(tCrC_r2sx(_, i + j), t);
2130         cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2131     }
2132     __syncthreads();
2133
2134     // S→G: shared memory 写回 global memory
2135     #pragma unroll
2136     for (int j = 0; j < step; ++j) {
2137         cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2138     }
2139     __syncthreads();
2140 }
2141 }

```

```

2142 }
2143
2144 // =====
2145 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2146 // =====
2147 // CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
2148 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2149 // kernel 接收这些类型的实例 (通常为 struct), 在编译期完成全部映射推导。
2150 //
2151 // 面试常问: 「CuTe 为什么比手写 PTX 简洁?」
2152 // 答: 手写需要:
2153 //   1) 手动计算每线程的 smem 地址偏移
2154 //   2) 手动计算 ldmatrix 的 lane 映射
2155 //   3) 手动插入 swizzle XOR 到每个地址计算点
2156 //   4) 手动做 epilogue 的 warp shuffle 编排
2157 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2158 // 在编译期自动完成上述全部推导, 生成与手写等价的 PTX 指令序列。
2159 //
2160 // 模板参数推导链 (面试速记):
2161 //   SM80_16x8x16_F16F16F16F16_TN ← MMA 指令 atom
2162 //     → MMA_Atom<MMA_Traits<mma_op>>
2163 //     → make_tiled_mma(mma_atom, EURepeat{2,2,1}, ValTile{32,32,16})
2164 //     → TiledMMA (128 threads, 4 warps × 2×2 slices)
2165 //
2166 //   SM80_CP_ASYNC_CACHEGLOBAL<uint128_t> ← G→S copy atom
2167 //     → Copy_Atom<Copy_Traits<g2s_copy_op>, T>
2168 //     → make_tiled_copy(copy_atom, ThrLayout{32,4}, ValLayout{1,8})
2169 //     → G2SCopy: 32×4=128 threads, each copies 1×8=8 halves = 128 bits
2170 //
2171 //   Swizzle<3,3,3> + Atom{8,BK} → composition → tile_to_shape{BM,BK,kStage}
2172 //     → SmemLayoutA/B: 带 XOR swizzle 的 smem 排布
2173 //
2174 //   SM75_U32x4_LDSM_N ← S→R copy atom (ldmatrix 的 CuTe 封装)
2175 //     → Copy_Atom<Copy_Traits<s2r_copy_op>, T>
2176 //
2177 //   UniversalCopy<uint128_t> ← S→G copy atom (128-bit store)
2178 //     → make_tiled_copy(copy_atom, ThrLayout{32,4}, ValLayout{1,8})
2179 //
2180 // ★ Tile 尺寸速查:
2181 //   BM=128: M 方向 block tile (16 × 2 EU × 4 Val = 128)
2182 //   BN=256: N 方向 block tile (8 × 2 EU × 16 Val... etc. 实际 8 × 2 × 16 = 256)
2183 //   BK=32: K 方向 block tile (= kMmaPK = 1 × 1 × 16 = 16? 不对, BK=32 由 SmemLayout 控制)
2184 //   kStage=2: 双缓冲 pipeline
2185 // =====
2186 template <typename T, const int Stages = 2>
2187 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2188     using namespace cute;
2189
2190     auto BM = Int<128>{};
2191     auto BN = Int<256>{};
2192     auto BK = Int<32>{};
2193     auto KStage = Int<Stages>{};
2194     auto kSmemLayoutCBatch = Int<4>{}; // C smem pipeline depth
2195
2196     // SmemLayout: Swizzle<3,3,3> 做 512-value (1024-byte) XOR 置换消除 bank conflict
2197     // composition: 将 swizzle pattern 应用到 8×BK 的 base layout
2198     // tile_to_shape: 将 atom layout 扩展到完整的 BM×BK×kStage
2199     using SmemLayoutAtom = decltype(composition(
2200         Swizzle<3, 3, 3>{},
2201         make_layout(make_shape(Int<8>{}, Int<BK>{}),
2202             make_stride(Int<BK>{}, Int<1>{}))));
2203     using SmemLayoutA = decltype(tile_to_shape(
2204         SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));

```

```

2205 using SmemLayoutB = decltype(tile_to_shape(
2206     SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
2207
2208 // MMA: SM80_16x8x16_F16F16F16F16_TN
2209 // TN = A row-major, B col-major (与 Phase 7b 手写 MMA 完全相同的布局约定)
2210 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2211 using mma_traits = MMA_Traits<mma_op>;
2212 using mma_atom = MMA_Atom<mma_traits>;
2213 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2214
2215 static constexpr int kMmaEURepeatM = 2;
2216 static constexpr int kMmaEURepeatN = 2;
2217 static constexpr int kMmaEURepeatK = 1;
2218 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2219 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2220 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2221
2222 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2223     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2224 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2225 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2226
2227 // G2S Copy: cp.async 128-bit, 32x4 threads each loading 1x8 halves
2228 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2229 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2230 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2231 using G2SCopyA = decltype(make_tiled_copy(
2232     g2s_copy_atom{},
2233     make_layout(make_shape(Int<32>{}, Int<4>{}),
2234         make_stride(Int<4>{}, Int<1>{}))),
2235     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2236 using G2SCopyB = G2SCopyA;
2237
2238 // S2R Copy: ldmatrix (SM75_U32x4_LDSM_N), 由 make_tiled_copy_A/B 自动与 TiledMMA 对齐
2239 using s2r_copy_op = SM75_U32x4_LDSM_N;
2240 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2241 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2242 using S2RCopyAtomA = s2r_copy_atom;
2243 using S2RCopyAtomB = s2r_copy_atom;
2244
2245 // Epilogue smem layout for C: Swizzle<3,3,3> on 32x32 atom, 4 pipeline stages
2246 using SmemLayoutAtomC = decltype(composition(
2247     Swizzle<3, 3, 3>{},
2248     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2249         make_stride(Int<kMmaPN>{}, Int<1>{}))));
2250 using SmemLayoutC = decltype(tile_to_shape(
2251     SmemLayoutAtomC{},
2252     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2253
2254 // R2S Copy: 寄存器 → shared memory (epilogue 阶段, UniversalCopy<int> 做逐 int 拷贝)
2255 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2256
2257 // S2G Copy: shared memory → global (128-bit store)
2258 using S2GCopyAtomC = Copy_Atom<UniversalCopy<cute::uint128_t>, T>;
2259 using S2GCopyC = decltype(make_tiled_copy(
2260     S2GCopyAtomC{},
2261     make_layout(make_shape(Int<32>{}, Int<4>{}),
2262         make_stride(Int<4>{}, Int<1>{}))),
2263     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2264
2265 // Grid/Block 配置
2266 int BX = (N + BN - 1) / BN;
2267 int BY = (M + BM - 1) / BM;

```



```

2268 dim3 block(size(MMA{})); // 128 threads
2269 dim3 grid(BX, BY);
2270
2271 // Smem 大小: A tile + B tile (C epilogue 复用 A tile 空间)
2272 static constexpr int shm_size_AB =
2273     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2274 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2275 // C smem ≤ A 的一个 pipe, 可以安全复用
2276 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2277     "C shared memory must fit within one A pipe");
2278 static constexpr int kShmSize =
2279     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2280
2281 cudaFuncSetAttribute(
2282     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2283         SmemLayoutA, SmemLayoutB, SmemLayoutC,
2284         S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2285         S2GCopyAtomC, S2GCopyC>,
2286     cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2287
2288 hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2289     SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
2290     S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC>
2291     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2292 }
2293
2294 #endif /* NOTES_V2_ENABLE_CUTE */
2295
2296 // =====
2297 // Phase 7d: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
2298 // =====
2299 // 面试要点 (WGMMMA vs MMA 对比):
2300 // - MMA: warp 级 (32 threads), 同步执行
2301 // - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-forget)
2302 // - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
2303 // - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
2304 // - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
2305 // - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
2306
2307 // ---- WGMMMA 辅助函数 ----
2308 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7):
2309 // - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
2310 //   (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行)。
2311 // - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N)。
2312 //   N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3),
2313 //   都是 "wait until only N or fewer groups are pending"。
2314 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
2315 #define WGMMMA_COMMIT_GROUP() \
2316     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
2317 #define WGMMMA_WAIT_GROUP(n) \
2318     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
2319
2320 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
2321 // 编码规则 (PTX ISA §9.7.15.5.1.2.2):
2322 //   encoded = (x & 0x3FFFF) >> 4
2323 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
2324 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
2325 // 可省略以扩大可表示范围。
2326 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
2327 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
2328
2329 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
2330 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,

```



```

2331 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
2332 //
2333 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
2334 // -----
2335 // | 位域          | 范围      | 大小 | 含义
2336 // |-----|-----|-----|-----
2337 // | start_address  | [0, 14)   | 14   | smem 基址编码
2338 // | (unused)       | [14, 16)  | 2    | -
2339 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
2340 // | (unused)       | [30, 32)  | 2    | -
2341 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
2342 // | (unused)       | [46, 49)  | 3    | -
2343 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
2344 // | (unused)       | [52, 62)  | 10   | -
2345 // | layout_type    | [62, 64)  | 2    | swizzle 模式
2346 // -----
2347 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
2348 //
2349 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
2350 // - 128B swizzle atom 在 128-bit 单位下为 8×8
2351 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
2352 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
2353 // - 这是 stride_byte_offset=1024 的来源
2354 //
2355 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
2356 //   此处填 16 仅为占位, 编码后为 1。
2357 //
2358 // - base_offset=0 (bits [49,52) 未显式置位) , 表示 smem ptr 已 1024B 对齐。
2359 //   若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
2360 //
2361 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
2362 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
2363 __device__ inline uint64_t make_smem_desc(half *ptr) {
2364     // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
2365     // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
2366     uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2367
2368     // 从全零开始: 所有位域初始化为 0。
2369     uint64_t desc = 0x0000000000000000;
2370
2371     // — bits [0, 14): start_address —
2372     // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
2373     // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
2374     // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
2375     desc |= SMEM_DESC_ENCODE(addr);
2376
2377     // — bits [16, 30): leading_byte_offset —
2378     // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
2379     // << 16 将编码值放到 bits [16, 30)。
2380     // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
2381     // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
2382     // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
2383     //   对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
2384     //   对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
2385     desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
2386
2387     // — bits [32, 46): stride_byte_offset —
2388     // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
2389     // << 32 将编码值放到 bits [32, 46)。
2390     // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2) 。
2391     // 1024 的来源 (K-Major + 128B swizzle + half) :
2392     //   一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
2393     //   从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。

```

```

2394 // 若为 MN-Major: SB0 是从首列到下一列的偏移 (8 列一组)。
2395 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
2396
2397 // — bits [62, 64): layout_type (swizzle mode) —
2398 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
2399 // 1 = 128B swizzle mode。
2400 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
2401 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
2402 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
2403 desc |= 1llu << 62;
2404
2405 return desc;
2406 }
2407
2408 // ---- WGMMA PTX 指令宏 ----
2409 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
2410 // m64n128k16: M=64, N=128, K=16。一次 WGMMA 计算 D[64×128] += A[64×16] × B[16×128]。
2411 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
2412 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
2413 // 每线程 64 half = 32 uint32。
2414 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
2415 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
2416 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
2417 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
2418 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
2419 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
2420                                     TransB) \
2421 { \
2422     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
2423     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
2424     asm volatile( \
2425         "{\n" \
2426         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
2427         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
2428         "%8, %9, %10, %11, %12, %13, %14, %15, " \
2429         "%16, %17, %18, %19, %20, %21, %22, %23, " \
2430         "%24, %25, %26, %27, %28, %29, %30, %31}, " \
2431         "%32, " \
2432         "%33, " \
2433         "%34, %35, %36, %37, %38;\n" \
2434         "}\n" \
2435         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
2436         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
2437         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
2438         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
2439         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
2440         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
2441         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
2442         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
2443         : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
2444         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
2445         "n"(int32_t(TransB))); \
2446 }
2447
2448 // ---- TMA Shared Memory Layout ----
2449 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
2450 //
2451 // 每个 stage 包含两块 smem:
2452 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
2453 //   B tile: TMA 按 [BN, BK] 物理写入 (详见下方 ★), BN×BK 个 half
2454 //
2455 // ★ 关键区别 — WGMMA 的 B 物理存储 vs 逻辑视图:
2456 //

```

```

2457 // 上层数据: 源矩阵  $B^T$  [N, K] row-major (TN 布局, B 的列以行优先形式存储)。
2458 // 物理存储: TMA 将  $B^T$  的 tile 写入 smem, 物理布局为 [BN, BK] row-major
2459 // (BN=128 行, BK=64 列, 每行 64 个连续的 half)。
2460 // TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,
2461 // TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
2462 // 逻辑视图: WGMMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
2463 // 通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按 [BK, BN] 寻址。
2464 // 实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
2465 // swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
2466 //
2467 // stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
2468 // 128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
2469 // stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
2470 // = 8 rows × (BK=64) halves × 2 bytes = 1024。
2471 // 这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
2472 //
2473 // 与 MMA(TN) 的对比:
2474 // MMA (TN): smem 存  $B^T$  [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
2475 // WGMMMA: smem 物理存 [BN, BK] (TMA 直写), WGMMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
2476 // 简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
2477 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
2478     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
2479     // B tile 标记:
2480     // - 物理存储: TMA 从  $B^T$ [N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
2481     // - 逻辑视图: WGMMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
2482     // 以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
2483     // - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN),
2484     // 但写 BN×BK 更能体现物理存储形态
2485     alignas(128) half B[BN * BK * QSIZE];
2486 };
2487
2488 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
2489 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化) :
2490 //
2491 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
2492 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
2493 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
2494 //
2495 // WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
2496 // 将 A/B tile 从 HBM 搬到 shared memory。
2497 // WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
2498 //
2499 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
2500 // - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪, 可被使用
2501 // - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可以被覆盖
2502 //
2503 // 本节重点理解:
2504 // 1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
2505 // 即可搬运整个 2D tile, 无需所有线程参与。
2506 // 2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
2507 // 3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
2508 // Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
2509 //
2510 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比) :
2511 // WGMMMA Atom: m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
2512 // K Tile: BK=64, 每个 K tile 包含 BK/WGMMMA_K=4 个 WGMMMA atom
2513 // M Tile: BM=128, 每个 M tile 包含 BM/WGMMMA_M=2 个 WGMMMA atom
2514 // N Tile: BN=128, 单个 WGMMMA_N=128 即可覆盖, 无需在 N 方向分块
2515 // Block Tile: C[128,128] = A[128,64] × B[64,128]
2516 // Threads: 256 = 2 warpgroups × 128 threads/warpgroup
2517 // Producer(WG0): 128 threads (仅 thread 0 做 TMA 提交)
2518 // Consumer(WG1): 128 threads = 4 warps (全部参与 WGMMMA 和写回)
2519 //

```

```

2520 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
2521 // 的延迟被计算完全掩盖。
2522 //
2523 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
2524 // - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
2525 // Block: (256, 1, 1), 2 warpgroups
2526 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
2527 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
2528           const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
2529           const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
2530           const bool BLOCK_SWIZZLE = false>
2531 __global__ void __launch_bounds__(NUM_THREADS)
2532   hgemm_wgmma_stages_tn(
2533     int M, int N, int K, half *C,
2534     const UTensorMap *__restrict__ tensorMapA,
2535     const UTensorMap *__restrict__ tensorMapB) {
2536   // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
2537   // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
2538   // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
2539   // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
2540   // 不展开宿主侧 create_tensor_map 细节。
2541
2542   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
2543   // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
2544   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
2545   const int by = blockIdx.y;
2546   // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
2547   // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
2548   constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
2549   // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
2550   // 当只有一个 consumer 时, 它负责全部 BM=128 行
2551   constexpr int B_WG_M = BM / num_consumers; // 128
2552
2553   // 边界检查: 确保当前 block 不超出 M/N 范围
2554   if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
2555     return;
2556
2557   // ---- Shared Memory 分配 ----
2558   // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
2559   // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
2560   extern __shared__ __align__(128) uint8_t smem_AB[];
2561   WgmmaSMem<BM, BN, BK, K_STAGE> &s =
2562     *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE>*>(smem_AB);
2563   half *s_a = s.A;
2564   half *s_b = s.B;
2565
2566   // ---- cuda::barrier 初始化 ----
2567   //
2568   // ★ Barrier 机制速查 (arrive / wait 核心语义):
2569   //
2570   // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
2571   //
2572   //   init(bar, N): 设置 arrive_count = N。
2573   //   含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
2574   //
2575   //   arrive(): 线程声明"我已到达此 barrier 点"。
2576   //   - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
2577   //   - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
2578   //
2579   //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
2580   //   - 即: 等待 phase 翻转。
2581   //   - Phase 翻转条件: (1) arrive 次数达到 arrive_count
2582   //                     (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完

```

```

2583 //
2584 // 典型用法: b.wait(b.arrive())
2585 // - arrive() 注册自己到达, 拿到 token (当前 phase = P)
2586 // - wait(token) 阻塞直到 phase 翻转 (P → P+1)
2587 // - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
2588 // - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
2589 //
2590 // ★ 本 kernel 的 Pipeline 同步协议 (K_STAGE=3, arrive_count=129) :
2591 //
2592 // Producer (1 thread)          Consumer (128 threads)
2593 // -----
2594 //                                C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
2595 // ┌ for each k_tile: ─┐          ┌ for each k_tile: ─┐
2596 // │ P1: arrive+wait(empty[q]) │ │ C1: arrive+wait(full[q]) │
2597 // │   ↓ 等 stage q 被消费完   │ │   ↓ 等 TMA 数据就绪     │
2598 // │ P2: TMA(A[q]) + TMA(B[q]) │ │ C2: WGMMMA 计算         │
2599 // │   ↓ 异步拷贝              │ │ C3: wait WGMMMA 完成     │
2600 // │ P3: arrive_tx(full[q])    │ │ C4: arrive(empty[q])    │
2601 // │   ↑ 通知: stage q 数据就绪 │ │   ↑ 通知: stage q 已消费完 │
2602 // └──────────────────┘          └──────────────────┘
2603 //
2604 // 关键不变式 (invariant) :
2605 // - Producer 不会覆盖 Consumer 正在读的 stage
2606 // - Consumer 不会读 Producer 还没写完的 stage
2607 // - 通过 full/empty 两个 barrier 的 phase 交替来保证
2608 //
2609 // 每个 stage 有两个 barrier:
2610 //   full[qidx]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
2611 //   empty[qidx]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
2612 //
2613 // arrive_count = 128 (consumer) + 1 (producer) = 129:
2614 // 每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
2615 #pragma nv_diag_suppress static_var_with_dynamic_init
2616 __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
2617 __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
2618 #pragma nv_diag_default static_var_with_dynamic_init
2619
2620 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
2621 const int num_blocks_k = K / BK;
2622 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
2623 const int tid = threadIdx.x % 128;   // 0~127 within warpgroup
2624
2625 // 初始化 barriers (仅 thread 0 执行)
2626 if (threadIdx.x == 0) {
2627     for (int i = 0; i < K_STAGE; ++i) {
2628         // arrive_count = 129: 128 consumer + 1 producer
2629         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
2630         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
2631         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
2632         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
2633         init(&empty[i], num_consumers * 128 + 1); // same
2634     }
2635     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
2636     // 对 async proxy (TMA/WGMMMA) 可见。
2637     cuda::device::experimental::fence_proxy_async_shared_cta();
2638 }
2639 __syncthreads();
2640
2641 // =====
2642 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
2643 // =====
2644 //
2645 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工, 不是先后顺序:

```



```

2646 // - WG0 (threadIdx.x 0~127): 全部走 if 分支, 做 Producer。
2647 // - WG1 (threadIdx.x 128~255): 全部走 else 分支, 做 Consumer。
2648 //
2649 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp, 两者**同时**推进:
2650 // Producer: for (k_iter = 0 .. num_blocks_k) { P1→P2→P3 } ← 自己的循环
2651 // Consumer: for (k_iter = 0 .. num_blocks_k) { C1→C2→C3→C4 } ← 自己的循环
2652 //
2653 // 两个 warpgroups 各自独立迭代 K-tiles, 通过 full[] / empty[] barrier 同步:
2654 // - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞, 等 Consumer 消费完
2655 // - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞, 等 TMA 搬运完
2656 //
2657 // 这是生产者-消费者模型的 GPU 实现: 不需要共享循环变量, barrier 的 phase
2658 // 交替天然保证了流水线串行化 (at most K_STAGE-1 steps ahead)。
2659 // =====
2660 // Producer Warpgroup (WG0, threadIdx.x 0~127)
2661 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
2662 // 只有 tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
2663 // =====
2664 if (wg_idx == 0) {
2665     if (tid == 0) {
2666         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
2667         int qidx = 0;
2668         for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
2669             if (qidx == K_STAGE)
2670                 qidx = 0;
2671
2672             // Step P1: 等待 stage qidx 变为"空" (可被覆盖写入)
2673             //
2674             // 模式 empty[qidx].wait(empty[qidx].arrive()):
2675             // - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达,
2676             //   获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
2677             //   128 次 arrive (来自上一轮迭代或 C0 初始化), 则加上这次共 129 次
2678             //   → phase 立即翻转。
2679             // - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
2680             //   由于 arrive() 是第 129 次到达, phase 在 arrive 时已翻转,
2681             //   wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
2682             //
2683             // 语义: Producer 说"我准备好写 stage qidx 了, Consumer 用完没有?"
2684             //   如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
2685             //   如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
2686             empty[qidx].wait(empty[qidx].arrive());
2687
2688             // Step P2: 提交 TMA 2D 拷贝指令
2689             // cp_async_bulk_tensor_2d_global_to_shared:
2690             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
2691             //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
2692             //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
2693             //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
2694             //
2695             // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
2696             // 无需线程逐元素搬运, 零寄存器开销。
2697             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
2698             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
2699                 &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
2700                 full[qidx]);
2701
2702             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
2703             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
2704                 &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
2705                 full[qidx]);
2706
2707             // Step P3: 通知 Consumer: stage qidx 的 TMA 数据已就绪
2708             //

```

```

2709 // barrier_arrive_tx(bar, arrive_count_update, byte_count):
2710 // - 在 bar 上注册 1 次到达 (arrive_count_update=1)
2711 // - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
2712 // - Phase 翻转条件:
2713 // (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
2714 // (b) 所有声明的 async 字节已写入 smem
2715 // 两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
2716 // - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
2717 [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
2718     full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
2719 }
2720 }
2721 }
2722 // =====
2723 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
2724 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
2725 // 所有 128 个线程 (4 warps) 全部参与。
2726 // =====
2727 else {
2728     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
2729     //
2730     // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
2731     // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[qidx].wait()
2732     // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
2733     //
2734     // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
2735     // Producer 后续的 empty[qidx].arrive() 作为第 129 次, 触发 phase 翻转。
2736     for (int i = 0; i < K_STAGE; ++i) {
2737         [[maybe_unused]] auto token = empty[i].arrive();
2738     }
2739
2740     // 累加器寄存器声明
2741     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:
2742     // - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
2743     // - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
2744     // - d[*][g][*]: N 方向第 g 组 16 列
2745     // - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
2746     // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
2747     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
2748     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
2749
2750     int qidx = 0;
2751     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
2752     for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
2753         if (qidx == K_STAGE)
2754             qidx = 0;
2755
2756         // Step C1: 等待 TMA 数据就绪 (full 信号)
2757         //
2758         // 模式 full[qidx].wait(full[qidx].arrive()):
2759         // - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
2760         // - 当前 phase 累积: 128/129, phase 尚未翻转。
2761         // - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
2762         //   贡献第 129 次到达 + TMA 字节全部写完 -> phase 翻转 -> wait 返回。
2763         //
2764         // 语义: Consumer 说"我准备好读 stage qidx 了, 数据到了没有? "
2765         full[qidx].wait(full[qidx].arrive());
2766
2767         // Step C2: 发射 WGMMMA 指令序列
2768         //
2769         // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
2770         // 标准流程: FENCE -> 发射 WGMMMA -> COMMIT -> WAIT。
2771         //

```



```

2772 // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
2773 // - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
2774 // - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
2775 // - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
2776 //
2777 // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
2778 // D[64×128] += A[64×16] × B[16×128]
2779 // A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
2780 // Scaled=1: 累加。K 维有 BK/WGMMMA_K=4 次迭代, 每次都用 Scaled=1。
2781 WGMMMA_FENCE();
2782
2783 // M 维迭代: BM/WGMMMA_M = 128/64 = 2 个 WGMMMA atom
2784 // 每个 atom 处理 64 行 M 方向数据。
2785 #pragma unroll
2786 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
2787     // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
2788     // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
2789     half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
2790
2791     // K 维迭代: BK/WGMMMA_K = 64/16 = 4 次 WGMMMA (累加)
2792     // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
2793     #pragma unroll
2794     for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
2795         // 第 k_it 次 WGMMMA:
2796         // A: wgmma_sA + k_it * WGMMMA_K (A 的 K 维起始位置)
2797         // B: s_b + qidx * BK * BN + k_it * WGMMMA_K (B 的 K 维起始位置)
2798         // 注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
2799         // 第 k_it*16 行开始取 16 行, 每行 BN=128 列。
2800         // Scaled=1: 累加 (K 维迭代需要累积结果)
2801         WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMMA_K,
2802                                     s_b + qidx * BK * BN + k_it * WGMMMA_K,
2803                                     1, // Scaled=1: accumulate (不清零)
2804                                     1, 1, 0, 0);
2805     }
2806 }
2807
2808 // Step C3: 提交并等待 WGMMMA 完成
2809 //
2810 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
2811 // 将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
2812 // 类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
2813 //
2814 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
2815 // 阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
2816 // * 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
2817 // 两者都是 "wait until only N or fewer of the most recent groups are pending"。
2818 // 此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
2819 // 必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
2820 WGMMMA_COMMIT_GROUP();
2821 WGMMMA_WAIT_GROUP(0);
2822
2823 // Step C4: 释放 stage qidx — 通知 Producer 可以覆盖写入
2824 //
2825 // 128 个 Consumer 线程在 empty[qidx] 上 arrive(), 为 **下一 phase** 积累到达次数。
2826 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
2827 //
2828 // 语义: Consumer 说"stage qidx 我已经读完了, 你可以放心覆盖了。"
2829 [[maybe_unused]] auto token = empty[qidx].arrive();
2830 }
2831
2832 // =====
2833 // Epilogue: 将寄存器中的累加结果写回 global memory C
2834 // =====

```

```

2835 //
2836 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
2837 //
2838 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
2839 // 这些 half 分布在 d[2][8][4] 数组中:
2840 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
2841 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
2842 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
2843 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
2844 //
2845 // 其中:
2846 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
2847 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
2848 //
2849 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
2850 // +-----+-----+
2851 // | reg[0] | reg[2] | <- rows [row, row+8)
2852 // |(col,+2)|(col+8)|
2853 // +-----+-----+
2854 // | reg[1] | reg[3] | <- rows [row+8, row+16)
2855 // |(col,+2)|(col+8)|
2856 // +-----+-----+
2857 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
2858 //
2859 // 三维遍历:
2860 // m_it=0: rows 0~63, m_it=1: rows 64~127
2861 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
2862 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
2863 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
2864
2865 const int lane = tid % 32;
2866 const int warp = tid / 32;
2867 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
2868 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
2869 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
2870 // 分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
2871 // 因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
2872 // 同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
2873 const int row = warp * 16 + lane / 4;
2874 // block_C: 当前 C tile 在 global memory 中的起始地址
2875 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
2876 half *block_C = C + by * BM * N + bx * BN;
2877
2878 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
2879 #pragma unroll
2880 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
2881     int yo = m_it * WGMMMA_M; // M 方向行偏移 (0 或 64)
2882     #pragma unroll
2883     for (int g = 0; g < WGMMMA_N / 16; ++g) {
2884         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
2885         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
2886         // 左上象限: (row, col)
2887         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
2888             d[m_it][g][0];
2889         // 左下象限: (row+8, col)
2890         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
2891             d[m_it][g][1];
2892         // 右上象限: (row, col+8)
2893         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
2894             d[m_it][g][2];
2895         // 右下象限: (row+8, col+8)
2896         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
2897             d[m_it][g][3];

```

```

2898     }
2899 }
2900 }
2901 }
2902
2903 #if (defined(NOTES_V2_ENABLE_WGMMA))
2904 // ---- Host-side TMA Tensor Map helpers (WGMMA test) ----
2905 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
2906 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
2907 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
2908 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
2909 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
2910 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
2911 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
2912 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMA 的 128B swizzle
2913 //
2914 // ★ gmem_prob_stride + 1 的含义:
2915 // 语法层面 — 数组名退化 + 指针算术:
2916 //   gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
2917 //   (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
2918 //   gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
2919 // 语义层面 — 跳过隐式的最内维 stride:
2920 //   cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
2921 //   stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
2922 //   固定等于 sizeof(element_type), 不需要显式传入。
2923 //   对于 tensorRank=2 的 FP16 矩阵:
2924 //     dim 0 (内维, K): stride = sizeof(half) ← 隐式, API 不需要
2925 //     dim 1 (外维, M): stride = sizeof(half)*K ← 需要传给 API
2926 //   gmem_prob_stride 数组的布局是内维在前:
2927 //     [0] = sizeof(half) ← 内维, 不传
2928 //     [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
2929 //   所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
2930 //
2931 // ★ smem_box_stride 为什么不需要 +1?
2932 //   与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
2933 //   最内维 stride 也必须显式传入 (不隐式)。
2934 //   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
2935 //   都是连续存放的, 维度间没有 padding/stride。
2936 //   对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
2937 //   即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
2938 //   所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
2939 //   两个 stride 值, 不需要跳过任何一个。
2940 //
2941 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
2942
2943 template <int BlockMajorSize, int BlockMinorSize>
2944 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
2945                                               half *gmem_ptr,
2946                                               int blocks_height,
2947                                               int blocks_width) {
2948     void *gmem_address = (void *)gmem_ptr;
2949     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
2950                                     (uint64_t)BlockMajorSize * blocks_height,
2951                                     1, 1, 1};
2952     uint64_t gmem_prob_stride[5] = {
2953         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
2954     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
2955                                    uint32_t(BlockMajorSize), 1, 1, 1};
2956     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
2957     CUresult result = cuTensorMapEncodeTiled(
2958         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
2959         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
2960         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,

```

```

2961     CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
2962     if (result != CUDA_SUCCESS)
2963         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
2964 }
2965
2966 __host__ static inline CUTensorMap *allocate_and_create_tensor_map(
2967     half *src, int blocks_height, int blocks_width) {
2968     CUTensorMap *tma_map_d;
2969     cudaMalloc(&tma_map_d, sizeof(CUTensorMap));
2970     CUTensorMap tma_map_host;
2971     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
2972     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUTensorMap),
2973         cudaMemcpyHostToDevice);
2974     return tma_map_d;
2975 }
2976 #endif /* NOTES_V2_ENABLE_WGMMA */
2977
2978 // =====
2979 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
2980 // =====
2981 // 面试题点 (FlashAttention 算法) :
2982 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
2983 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
2984 // 2. FA 三板斧:
2985 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqLen 分块 [Bc,d]
2986 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
2987 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
2988 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
2989 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
2990 //    - 优点: 减少 warp 间通信和 shuffle
2991 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
2992 //    for each K,V tile:
2993 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
2994 //        m_new = max(m_old, row_max(S_cur * scale))
2995 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
2996 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
2997 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
2998 //        O_final = O_new / l_final
2999 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
3000 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
3001 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
3002 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
3003 //    - 这是此实现的寄存器布局复用技巧, 不要背成 “所有 MMA A/C fragment
3004 //      都天然同构” 的通用结论
3005 //
3006 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
3007 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
3008 // Grid: ((QKV_seqLen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
3009 // Block: (128, 1, 1), kNumThreads=WARP_SIZE×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
3010 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
3011
3012 // ---- 寄存器填充辅助函数 ----
3013 template <typename T, int M, const int N, const int K = 2>
3014 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
3015     #pragma unroll
3016     for (int i = 0; i < M; ++i)
3017         #pragma unroll
3018         for (int j = 0; j < N; ++j)
3019             #pragma unroll
3020             for (int k = 0; k < K; ++k)
3021                 R[i][j][k] = val;
3022 }
3023

```

```

3024 template <typename T, int M, const int N = 2>
3025 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
3026     #pragma unroll
3027     for (int i = 0; i < M; ++i)
3028     #pragma unroll
3029         for (int j = 0; j < N; ++j)
3030             R[i][j] = val;
3031 }
3032
3033 // =====
3034 // FlashAttention-2 Split-Q Kernel (完整实现)
3035 // =====
3036 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
3037 //
3038 // Tile 设计 (以 kHeadDim=64 为例) :
3039 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
3040 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
3041 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
3042 //
3043 // 执行流程:
3044 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
3045 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
3046 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
3047 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMMA16816
3048 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
3049 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
3050 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
3051 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
3052 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
3053 //   8) 最终 rescale: O_final = (1/l_final) * O_final
3054 //   9) Epilogue: warp shuffle + 128-bit collective store
3055
3056 template <
3057     const int kHeadDim,           // head dim: 32, 64, 128
3058     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
3059     const int kMmaAtomN,          // 8 (MMA instruction N dimension)
3060     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
3061     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
3062     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
3063     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
3064     const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
3065     const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
3066     const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
3067     const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
3068     const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
3069     const int kStage,             // pipeline stages for K: 1 or 2
3070     const int kPad>              // padding for bank conflict avoidance
3071 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)
3072     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
3073                                     int QKV_seqlen, int QKV_head) {
3074     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
3075     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
3076     constexpr int kNumThreads = WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
3077     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
3078     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
3079     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
3080     const float scale = 1.0f / sqrtf((float)kHeadDim);
3081
3082     // Block indexing
3083     const int QKV_batch_id = blockIdx.y / QKV_head;
3084     const int QKV_head_id = blockIdx.y % QKV_head;
3085     const int Q_tile_id = blockIdx.x;
3086     const int tid = threadIdx.x;

```

```

3087 const int warp_id = tid / WARP_SIZE;
3088 const int lane_id = tid % WARP_SIZE;
3089 const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
3090 const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
3091
3092 // Global memory base offsets for this (batch, head)
3093 // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
3094 const int Q_gmem_offset =
3095     (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
3096     kHeadDim;
3097 const int K_gmem_offset = Q_gmem_offset;
3098 const int V_gmem_offset = Q_gmem_offset;
3099 const int O_gmem_offset = Q_gmem_offset;
3100
3101 // Thread-to-smem mapping for cooperative load
3102 int load_smem_Q_Br = tid / (kNumThreads / Br);
3103 int load_smem_Q_d =
3104     (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
3105 int load_smem_K_Bc = tid / (kNumThreads / Bc);
3106 int load_smem_K_d =
3107     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3108 int load_smem_V_Bc = tid / (kNumThreads / Bc);
3109 int load_smem_V_d =
3110     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3111
3112 int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
3113 if (load_gmem_Q_Br >= QKV_seqlen)
3114     return;
3115
3116 // ---- Shared memory layout ----
3117 extern __shared__ half smem[];
3118 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
3119 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
3120 half *Q_tile_smem = smem;
3121 half *K_tile_smem = Q_tile_smem + Q_tile_size;
3122 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
3123 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
3124 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
3125
3126 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
3127 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
3128 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
3129
3130 // ---- Online Softmax persistent state ----
3131 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
3132 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
3133 float lane_block_row_max_old[kValTileSeqLenQ][2];
3134 float lane_block_row_sum_old[kValTileSeqLenQ][2];
3135 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
3136 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
3137
3138 // ---- Register allocation ----
3139 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
3140 uint32_t R_K[kValTileSeqLenK][2]; // K regs
3141 uint32_t R_V[kValTileHeadDimV][2]; // V regs
3142 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
3143 // 对单个 m16n8k16 tile 而言:
3144 //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
3145 //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
3146 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
3147 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
3148 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
3149 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]

```



```

3150         [2]; // 0 accumulator (final output)
3151
3152     fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3153     fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
3154
3155     // =====
3156     // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
3157     // =====
3158     {
3159         int load_gmem_Q_addr =
3160             Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
3161         uint32_t load_smem_Q_ptr =
3162             smem_Q_base_ptr +
3163             (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
3164 #pragma unroll
3165         for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
3166             CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
3167         }
3168         CP_ASYNC_COMMIT_GROUP();
3169     }
3170
3171     // =====
3172     // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
3173     // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
3174     // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
3175     // =====
3176     if constexpr (kStage > 1) {
3177 #pragma unroll
3178         for (int stage = 0; stage < (kStage - 1); ++stage) {
3179             int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
3180             int load_gmem_K_addr =
3181                 K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3182             uint32_t load_smem_K_ptr =
3183                 smem_K_base_ptr +
3184                 (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3185                  load_smem_K_d) *
3186                 sizeof(half);
3187 #pragma unroll
3188             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3189                 CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3190             }
3191             CP_ASYNC_COMMIT_GROUP();
3192         }
3193         CP_ASYNC_WAIT_GROUP(kStage - 2);
3194         __syncthreads();
3195     }
3196
3197     // =====
3198     // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
3199     // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
3200     // =====
3201 #pragma unroll 1
3202     for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
3203         int smem_sel = tile_K_seqlen % kStage;
3204         int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
3205
3206         // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
3207         if constexpr (kStage > 1) {
3208             // 只有 kStage>1 才能真正做 K 的 pipeline:
3209             // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
3210             // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
3211             // Load current V tile (no pipeline for V — one stage is enough)
3212             {

```



```

3213     int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
3214     int load_gmem_V_addr =
3215         V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
3216     uint32_t load_smem_V_ptr =
3217         smem_V_base_ptr +
3218         (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
3219 #pragma unroll
3220     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3221         CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
3222     }
3223     CP_ASYNC_COMMIT_GROUP();
3224 }
3225
3226 // Prefetch next K tile (pipelined)
3227 if ((tile_K_seqlen + 1) < Tc) {
3228     int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
3229     int load_gmem_K_addr =
3230         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3231     uint32_t load_smem_K_ptr =
3232         smem_K_base_ptr +
3233         (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3234          load_smem_K_d) *
3235         sizeof(half);
3236 #pragma unroll
3237     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3238         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3239     }
3240     CP_ASYNC_COMMIT_GROUP();
3241 }
3242 }
3243
3244 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
3245 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3246 #pragma unroll
3247 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
3248     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
3249     #pragma unroll
3250     for (int i = 0; i < kValTileSeqLenQ; ++i) {
3251         int warp_smem_Q_Br =
3252             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
3253         int lane_smem_Q_Br =
3254             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
3255         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
3256         uint32_t lane_smem_Q_ptr =
3257             smem_Q_base_ptr +
3258             (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
3259         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
3260                    lane_smem_Q_ptr);
3261     }
3262
3263     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
3264     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
3265     #pragma unroll
3266     for (int j = 0; j < kValTileSeqLenK; ++j) {
3267         int warp_smem_K_Bc =
3268             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
3269         int lane_smem_K_Bc =
3270             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
3271         int lane_smem_K_d =
3272             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
3273         uint32_t lane_smem_K_ptr =
3274             smem_K_base_ptr +
3275             (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +

```

```

3276         lane_smem_K_d) *
3277         sizeof(half);
3278     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
3279 }
3280
3281 // MMA: S[tile] += Q[tile] @ K^T[tile]
3282 #pragma unroll
3283 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3284     #pragma unroll
3285     for (int j = 0; j < kValTileSeqLenK; ++j) {
3286         HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
3287                 R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
3288                 R_S[i][j][1]);
3289     }
3290 }
3291 } // end loop over d
3292 __syncthreads();
3293
3294 // =====
3295 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
3296 // =====
3297 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
3298 //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
3299 //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
3300 //   - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
3301 //   - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
3302 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
3303 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
3304 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
3305 // kValTileSeqLenK tiles.
3306
3307 float lane_row_max_new[kValTileSeqLenQ][2];
3308 float lane_row_sum_new[kValTileSeqLenQ][2];
3309 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
3310 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
3311
3312 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
3313 #pragma unroll
3314 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3315     #pragma unroll
3316     for (int j = 0; j < kValTileSeqLenK; ++j) {
3317         // Extract half values from R_S registers (C matrix fragment layout)
3318         float2 t_reg_S_0 =
3319             __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
3320         float2 t_reg_S_1 =
3321             __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
3322         // S = (Q@K^T) * scale
3323         float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
3324         float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
3325         lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
3326         lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
3327     }
3328     // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
3329     // {0,4,8,...,28} hold valid data)
3330     lane_row_max_new[i][0] =
3331         warp_reduce_max<4, float>(lane_row_max_new[i][0]);
3332     lane_row_max_new[i][1] =
3333         warp_reduce_max<4, float>(lane_row_max_new[i][1]);
3334 }
3335
3336 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
3337 // 面试关键点: 这里将 P 写回 R_S 寄存器!
3338 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,

```

```

3339 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
3340 #pragma unroll
3341 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3342     // m_new = max(m_old, m_cur)
3343     float block_row_max_new_0 =
3344         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
3345     float block_row_max_new_1 =
3346         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
3347
3348 #pragma unroll
3349 for (int j = 0; j < kValTileSeqLenK; ++j) {
3350     float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
3351     float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
3352
3353     // P = exp(S * scale - m_new), 用 fma 保证精度
3354     t_reg_S_0.x =
3355         __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
3356     t_reg_S_0.y =
3357         __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
3358     t_reg_S_1.x =
3359         __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
3360     t_reg_S_1.y =
3361         __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
3362
3363     // Accumulate row sums
3364     lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
3365     lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
3366
3367     // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
3368     HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
3369     HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
3370 }
3371
3372 // Warp-level reduce sum (warp_size = 4, same as max)
3373 lane_row_sum_new[i][0] =
3374     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
3375 lane_row_sum_new[i][1] =
3376     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
3377 }
3378 __syncthreads();
3379
3380 // =====
3381 // 3d: P@V - P[Br,Bc] @ V[Bc,d] = 0[Br,d]
3382 // =====
3383 // Wait for V to be ready before computing P@V
3384 if constexpr (kStage > 1) {
3385     if ((tile_K_seqLen + 1) < Tc) {
3386         CP_ASYNC_WAIT_GROUP(1);
3387     } else {
3388         CP_ASYNC_WAIT_GROUP(0);
3389     }
3390 } else {
3391     CP_ASYNC_WAIT_GROUP(0);
3392 }
3393 __syncthreads();
3394
3395 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
3396
3397 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
3398 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
3399 #pragma unroll
3400 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
3401     // ldmatrix.x2.trans: load V[Bc,d] with transposition

```

```

3402 // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
3403 // transposed ldmatrix
3404 #pragma unroll
3405 for (int j = 0; j < kValTileHeadDimV; ++j) {
3406     int warp_smem_V_d =
3407         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3408     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
3409     int lane_smem_V_d = warp_smem_V_d;
3410     uint32_t lane_smem_V_ptr =
3411         smem_V_base_ptr +
3412         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
3413     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
3414 }
3415
3416 // P matrix layout in R_S[i][j][2]:
3417 // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
3418 // | 64x64 | warp_KV 0 |
3419 // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
3420 // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
3421 // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
3422 // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
3423 // tile_V_Bc selects which 16-column slice of P to use:
3424 // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
3425 // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
3426 // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
3427 // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
3428 // 对应的 MMA A fragment 布局可以这样记:
3429 // - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
3430 // - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
3431 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
3432 // 才能把 R_S 中的 P 直接喂给 HMMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
3433 // fragment 都无条件成立的通用结论。layout转换逻辑:
3434 // C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
3435 int w = tile_V_Bc * 2;
3436 #pragma unroll
3437 for (int i = 0; i < kValTileSeqLenP; ++i) {
3438     #pragma unroll
3439     for (int j = 0; j < kValTileHeadDimV; ++j) {
3440         HMMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
3441             R_S[i][w][0], R_S[i][w][1], R_S[i][w+1][0], R_S[i][w+1][1], // A fragment = P
3442             R_V[j][0], R_V[j][1], // B fragment = V
3443             R_0[i][j][0], R_0[i][j][1]); // C fragment output
3444     }
3445 }
3446 } // end for tile_V_Bc
3447 __syncthreads();
3448
3449 // =====
3450 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
3451 // =====
3452 // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 0 与 l 的 rescale
3453 #pragma unroll
3454 for (int i = 0; i < kValTileSeqLenP; ++i) {
3455     float block_row_max_new_0 = lane_row_max_new[i][0];
3456     float block_row_max_new_1 = lane_row_max_new[i][1];
3457     float block_row_sum_new_0 = lane_row_sum_new[i][0];
3458     float block_row_sum_new_1 = lane_row_sum_new[i][1];
3459
3460     float block_row_max_old_0 = lane_block_row_max_old[i][0];
3461     float block_row_max_old_1 = lane_block_row_max_old[i][1];
3462
3463     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
3464     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);

```

```

3465 // Handle first iteration: m_old = -inf, need to use m_new directly
3466 block_row_max_old_0 =
3467     (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
3468 block_row_max_old_1 =
3469     (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
3470
3471 float rescale_o_factor_0 =
3472     __expf(block_row_max_old_0 - block_row_max_new_0);
3473 float rescale_o_factor_1 =
3474     __expf(block_row_max_old_1 - block_row_max_new_1);
3475
3476 // Rescale O_old + Add P@V in one fused step
3477 #pragma unroll
3478 for (int j = 0; j < kValTileHeadDimV; ++j) {
3479     // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
3480     // reg[0] -> rows 0~7 的 {c0,c1}
3481     // reg[1] -> rows 8~15 的 {c2,c3}
3482     float2 t_reg_0_0 = __half2float2(HALF2(R_0[i][j][0]));
3483     float2 t_reg_0_1 = __half2float2(HALF2(R_0[i][j][1]));
3484     float2 t_reg_D_0 = __half2float2(HALF2(R_D[i][j][0]));
3485     float2 t_reg_D_1 = __half2float2(HALF2(R_D[i][j][1]));
3486
3487     // O_new = exp(m_old - m_new) * O_old + P@V (fused multiply-add)
3488     t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
3489     t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
3490     t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
3491     t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
3492
3493     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3494     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3495 }
3496
3497 // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
3498 float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
3499 float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
3500 lane_block_row_sum_old[i][0] = __fmaf_rn(
3501     rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
3502 lane_block_row_sum_old[i][1] = __fmaf_rn(
3503     rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
3504
3505 // Update m
3506 lane_block_row_max_old[i][0] = block_row_max_new_0;
3507 lane_block_row_max_old[i][1] = block_row_max_new_1;
3508 }
3509
3510 // Wait for next K tile to be ready in smem before next iteration
3511 if constexpr (kStage > 1) {
3512     if ((tile_K_seqlen + 1) < Tc) {
3513         CP_ASYNC_WAIT_GROUP(0);
3514     }
3515     __syncthreads();
3516 }
3517 } // end outer loop over K seqlen
3518 __syncthreads();
3519
3520 // =====
3521 // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
3522 // =====
3523 #pragma unroll
3524 for (int i = 0; i < kValTileSeqLenP; ++i) {
3525     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
3526     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);

```

```

3528 #pragma unroll
3529 for (int j = 0; j < kValTileHeadDimV; ++j) {
3530     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3531     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3532     t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
3533     t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
3534     t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
3535     t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
3536     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3537     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
3538 }
3539 }
3540
3541 // =====
3542 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
3543 // =====
3544 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
3545 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
3546 #pragma unroll
3547 for (int i = 0; i < kValTileSeqLenP; ++i) {
3548     #pragma unroll
3549     for (int j = 0; j < kValTileHeadDimV; ++j) {
3550         uint32_t R_Z[2][4];
3551         R_Z[0][0] = R_D[i][j][0];
3552         R_Z[1][0] = R_D[i][j][1];
3553         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
3554         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
3555         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
3556         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
3557         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
3558         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
3559
3560         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
3561         if (lane_id % 4 == 0) {
3562             int store_warp_regs_0_Br =
3563                 warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
3564             int store_lane_gmem_0_Br =
3565                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
3566             int store_warp_regs_0_d =
3567                 warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3568             int store_lane_gmem_0_d = store_warp_regs_0_d;
3569             int store_gmem_0_addr_0 = 0_gmem_offset +
3570                 (store_lane_gmem_0_Br + 0) * kHeadDim +
3571                 store_lane_gmem_0_d;
3572             int store_gmem_0_addr_1 = 0_gmem_offset +
3573                 (store_lane_gmem_0_Br + 8) * kHeadDim +
3574                 store_lane_gmem_0_d;
3575             // LDST128BITS = reinterpret_cast<float4*>
3576             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
3577                 *reinterpret_cast<float4*>(&R_Z[0][0]);
3578             *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
3579                 *reinterpret_cast<float4*>(&R_Z[1][0]);
3580         }
3581     }
3582 }
3583 }
3584
3585 // =====
3586 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
3587 // =====
3588
3589 static inline void check(cudaError_t err, const char *msg) {
3590     if (err != cudaSuccess) {

```



```

3591     fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
3592     exit(EXIT_FAILURE);
3593 }
3594 }
3595
3596
3597 static void test_block_reduce(int N) {
3598
3599     srand(42);
3600     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3601     for (int i = 0; i < N; i++)
3602         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3603
3604     // CPU reference: sum of all elements
3605     double ref = 0.0;
3606     for (int i = 0; i < N; i++) ref += (double)h_a[i];
3607
3608     float *d_a, *d_y;
3609     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
3610     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
3611
3612     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
3613     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
3614
3615     dim3 block(128);
3616     dim3 grid((N + 127) / 128);
3617     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
3618     check(cudaGetLastError(), "blockreduce launch");
3619     check(cudaDeviceSynchronize(), "blockreduce sync");
3620
3621     float result;
3622     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
3623
3624     float err = fabsf(result - (float)ref);
3625     printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
3626         err < 1e-2f ? "PASS" : "FAIL");
3627
3628     free(h_a);
3629     cudaFree(d_a); cudaFree(d_y);
3630 }
3631
3632
3633 static void test_dot(int N) {
3634
3635     srand(42);
3636     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3637     float *h_b = (float *)malloc((size_t)N * sizeof(float));
3638     for (int i = 0; i < N; i++) {
3639         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3640         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3641     }
3642
3643     // CPU reference
3644     double ref = 0.0;
3645     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
3646
3647     float *d_a, *d_b, *d_y;
3648     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
3649     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
3650     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
3651
3652     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
3653     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");

```

```

3654 check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
3655
3656 dim3 block(128);
3657 dim3 grid((N + 127) / 128);
3658 dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
3659 check(cudaGetLastError(), "dot launch");
3660 check(cudaDeviceSynchronize(), "dot sync");
3661
3662 float result;
3663 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
3664
3665 float err = fabsf(result - (float)ref);
3666 printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
3667     err < 1e-2f ? "PASS" : "FAIL");
3668
3669 // ---- Dot Vec4 ----
3670 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
3671 dim3 block_v4(32);
3672 dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
3673 check(cudaGetLastError(), "dot_vec4 launch");
3674 check(cudaDeviceSynchronize(), "dot_vec4 sync");
3675
3676 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
3677 float err_v4 = fabsf(result - (float)ref);
3678 printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
3679
3680 free(h_a); free(h_b);
3681 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
3682 }
3683
3684
3685 static void test_relu(int N) {
3686     srand(42);
3687     float *h_x = (float *)malloc((size_t)N * sizeof(float));
3688     float *h_y = (float *)malloc((size_t)N * sizeof(float));
3689     for (int i = 0; i < N; i++)
3690         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3691
3692     float *d_x, *d_y;
3693     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
3694     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
3695     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
3696
3697     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
3698     dim3 block256(256);
3699     dim3 grid256((N + 255) / 256);
3700     relu<<<grid256, block256>>>(d_x, d_y, N);
3701     check(cudaGetLastError(), "relu launch");
3702     check(cudaDeviceSynchronize(), "relu sync");
3703     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
3704     float max_err = 0.0f;
3705     for (int i = 0; i < N; i++) {
3706         float expected = fmaxf(0.0f, h_x[i]);
3707         float err = fabsf(h_y[i] - expected);
3708         if (err > max_err) max_err = err;
3709     }
3710     printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3711
3712     dim3 block64(64);
3713     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
3714     check(cudaGetLastError(), "relu_vec4 launch");
3715     check(cudaDeviceSynchronize(), "relu_vec4 sync");
3716     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");

```

```

3717 max_err = 0.0f;
3718 for (int i = 0; i < N; i++) {
3719     float expected = fmaxf(0.0f, h_x[i]);
3720     float err = fabsf(h_y[i] - expected);
3721     if (err > max_err) max_err = err;
3722 }
3723 printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3724
3725 free(h_x); free(h_y);
3726 cudaFree(d_x); cudaFree(d_y);
3727 }
3728
3729
3730 static void test_elementwise(int N) {
3731     srand(42);
3732     float *h_a = (float *)malloc((size_t)N * sizeof(float));
3733     float *h_b = (float *)malloc((size_t)N * sizeof(float));
3734     for (int i = 0; i < N; i++) {
3735         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3736         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3737     }
3738
3739     float *d_a, *d_b, *d_c;
3740     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
3741     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
3742     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
3743     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
3744     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
3745
3746     dim3 block256(256);
3747     dim3 grid256((N + 255) / 256);
3748     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
3749     check(cudaGetLastError(), "eadd launch");
3750     check(cudaDeviceSynchronize(), "eadd sync");
3751     float *h_c = (float *)malloc((size_t)N * sizeof(float));
3752     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
3753     float max_err = 0.0f;
3754     for (int i = 0; i < N; i++) {
3755         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
3756         if (err > max_err) max_err = err;
3757     }
3758     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3759
3760     dim3 block64(64);
3761     check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
3762     elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
3763     check(cudaGetLastError(), "eadd_vec4 launch");
3764     check(cudaDeviceSynchronize(), "eadd_vec4 sync");
3765     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
3766     max_err = 0.0f;
3767     for (int i = 0; i < N; i++) {
3768         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
3769         if (err > max_err) max_err = err;
3770     }
3771     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3772
3773     free(h_a); free(h_b); free(h_c);
3774     cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
3775 }
3776
3777
3778 static void test_histogram(int N) {
3779     srand(42);

```

```

3780 int BINS = 16;
3781 int *h_hist = (int *)calloc(BINS, sizeof(int));
3782 int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
3783 int *h_idx = (int *)malloc((size_t)N * sizeof(int));
3784 for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
3785 for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
3786
3787 int *d_idx, *d_hist;
3788 check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
3789 check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
3790 check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
3791 check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
3792
3793 dim3 block256(256);
3794 dim3 grid256((N + 255) / 256);
3795 histogram<<<grid256, block256>>>(d_idx, d_hist, N);
3796 check(cudaGetLastError(), "histogram launch");
3797 check(cudaDeviceSynchronize(), "histogram sync");
3798 check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
3799
3800 float max_err = 0.0f;
3801 for (int i = 0; i < BINS; i++) {
3802     float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
3803     if (err > max_err) max_err = err;
3804 }
3805 printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3806
3807 free(h_hist); free(h_hist_ref); free(h_idx);
3808 cudaFree(d_idx); cudaFree(d_hist);
3809 }
3810
3811
3812 static void test_merge_attn_states(int num_tokens, int num_heads,
3813                                   int head_size) {
3814
3815     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
3816     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
3817
3818     float *h_prefix_out = (float *)malloc(out_size);
3819     float *h_suffix_out = (float *)malloc(out_size);
3820     float *h_prefix_lse = (float *)malloc(lse_size);
3821     float *h_suffix_lse = (float *)malloc(lse_size);
3822     float *h_output_ref = (float *)malloc(out_size);
3823
3824     srand(42);
3825     for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
3826         h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3827         h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3828     }
3829     for (int i = 0; i < num_heads * num_tokens; i++) {
3830         h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
3831         h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
3832     }
3833
3834     // CPU reference: 与 kernel 完全相同的浮点计算
3835     for (int t = 0; t < num_tokens; t++) {
3836         for (int h = 0; h < num_heads; h++) {
3837             float p_lse = h_prefix_lse[h * num_tokens + t];
3838             float s_lse = h_suffix_lse[h * num_tokens + t];
3839             p_lse = isinf(p_lse) ? -INFINITY : p_lse;
3840             s_lse = isinf(s_lse) ? -INFINITY : s_lse;
3841
3842             float max_lse = fmaxf(p_lse, s_lse);

```

```

3843     p_lse -= max_lse;
3844     s_lse -= max_lse;
3845     float p_se = expf(p_lse);
3846     float s_se = expf(s_lse);
3847     float p_scale = p_se / (p_se + s_se);
3848     float s_scale = s_se / (p_se + s_se);
3849
3850     int head_off = t * num_heads * head_size + h * head_size;
3851     for (int d = 0; d < head_size; d++) {
3852         h_output_ref[head_off + d] =
3853             h_prefix_out[head_off + d] * p_scale +
3854             h_suffix_out[head_off + d] * s_scale;
3855     }
3856 }
3857 }
3858
3859 float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
3860 check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
3861 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
3862 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
3863 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
3864 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
3865
3866 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
3867                 cudaMemcpyHostToDevice),
3868        "merge_attn H2D p_out");
3869 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
3870                 cudaMemcpyHostToDevice),
3871        "merge_attn H2D s_out");
3872 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
3873                 cudaMemcpyHostToDevice),
3874        "merge_attn H2D p_lse");
3875 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
3876                 cudaMemcpyHostToDevice),
3877        "merge_attn H2D s_lse");
3878
3879 int threads_per_head = head_size / 4;
3880 int total_threads = num_tokens * num_heads * threads_per_head;
3881 dim3 block(128);
3882 dim3 grid((total_threads + 127) / 128);
3883 merge_attn_states<<<grid, block>>>(
3884     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
3885     num_tokens, num_heads, head_size);
3886 check(cudaGetLastError(), "merge_attn launch");
3887 check(cudaDeviceSynchronize(), "merge_attn sync");
3888
3889 float *h_output = (float *)malloc(out_size);
3890 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
3891        "merge_attn D2H");
3892
3893 float max_err = 0.0f;
3894 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
3895     float err = fabsf(h_output[i] - h_output_ref[i]);
3896     if (err > max_err) max_err = err;
3897 }
3898 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates", max_err,
3899        max_err < 1e-4f ? "PASS" : "FAIL");
3900
3901 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
3902 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
3903 h_suffix_lse[0] = 0.0f;
3904 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
3905                 cudaMemcpyHostToDevice),

```

```

3906     "merge_attn H2D inf lse");
3907 merge_attn_states<<<grid, block>>>{
3908     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
3909     num_tokens, num_heads, head_size);
3910 check(cudaGetLastError(), "merge_attn inf launch");
3911 check(cudaDeviceSynchronize(), "merge_attn inf sync");
3912 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
3913     "merge_attn inf D2H");
3914
3915 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )
3916 float inf_err = 0.0f;
3917 for (int d = 0; d < head_size; d++) {
3918     float err = fabsf(h_output[d] - h_suffix_out[d]);
3919     if (err > inf_err) inf_err = err;
3920 }
3921 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates-inf", inf_err,
3922     inf_err < 1e-4f ? "PASS" : "FAIL");
3923
3924 free(h_prefix_out);
3925 free(h_suffix_out);
3926 free(h_prefix_lse);
3927 free(h_suffix_lse);
3928 free(h_output_ref);
3929 free(h_output);
3930 cudaFree(d_prefix_out);
3931 cudaFree(d_suffix_out);
3932 cudaFree(d_prefix_lse);
3933 cudaFree(d_suffix_lse);
3934 cudaFree(d_output);
3935 }
3936
3937
3938 static void test_softmax(int N) {
3939     // Use N = blockDim.x so one block processes all elements as one token.
3940     constexpr int NUM_THREADS = 256;
3941     if (N != NUM_THREADS) {
3942         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
3943         return;
3944     }
3945
3946     srand(42);
3947     float *h_x = (float *)malloc((size_t)N * sizeof(float));
3948     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
3949     for (int i = 0; i < N; i++)
3950         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
3951
3952     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
3953     float max_val = -FLT_MAX;
3954     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
3955     double sum_exp = 0.0;
3956     for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
3957     for (int i = 0; i < N; i++)
3958         h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
3959
3960     float *d_x, *d_y;
3961     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
3962     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
3963
3964     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
3965
3966     dim3 block(256);
3967     dim3 grid(1); // one token covering all N elements
3968     online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);

```

```

3969 check(cudaGetLastError(), "softmax launch");
3970 check(cudaDeviceSynchronize(), "softmax sync");
3971
3972 float *h_y = (float *)malloc((size_t)N * sizeof(float));
3973 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
3974
3975 float max_err = 0.0f;
3976 for (int i = 0; i < N; i++) {
3977     float err = fabsf(h_y[i] - h_y_ref[i]);
3978     if (err > max_err) max_err = err;
3979 }
3980 printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3981
3982 // ---- Safe Softmax ----
3983 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3984 check(cudaGetLastError(), "safe_softmax launch");
3985 check(cudaDeviceSynchronize(), "safe_softmax sync");
3986 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
3987 max_err = 0.0f;
3988 for (int i = 0; i < N; i++) {
3989     float err = fabsf(h_y[i] - h_y_ref[i]);
3990     if (err > max_err) max_err = err;
3991 }
3992 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3993
3994 // ---- Naive Softmax ----
3995 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3996 check(cudaGetLastError(), "naive_softmax launch");
3997 check(cudaDeviceSynchronize(), "naive_softmax sync");
3998 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
3999 max_err = 0.0f;
4000 for (int i = 0; i < N; i++) {
4001     float err = fabsf(h_y[i] - h_y_ref[i]);
4002     if (err > max_err) max_err = err;
4003 }
4004 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4005
4006 free(h_x); free(h_y); free(h_y_ref);
4007 cudaFree(d_x); cudaFree(d_y);
4008 }
4009
4010
4011 static void test_rms_norm(int N, int K) {
4012
4013     srand(42);
4014     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4015     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4016     for (int i = 0; i < N * K; i++)
4017         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4018     float g = 1.5f; // gain
4019
4020     // CPU reference: y = (x / rms(x)) * g
4021     float epsilon = 1e-5f;
4022     for (int n = 0; n < N; n++) {
4023         double sum_sq = 0.0;
4024         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
4025         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
4026         for (int k = 0; k < K; k++)
4027             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
4028     }
4029
4030     float *d_x, *d_y;
4031     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");

```



```

4032 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
4033 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
4034
4035 dim3 block(128);
4036 dim3 grid(N);
4037 rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
4038 check(cudaGetLastError(), "rmsnorm launch");
4039 check(cudaDeviceSynchronize(), "rmsnorm sync");
4040
4041 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4042 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
4043
4044 float max_err = 0.0f;
4045 for (int i = 0; i < N * K; i++) {
4046     float err = fabsf(h_y[i] - h_y_ref[i]);
4047     if (err > max_err) max_err = err;
4048 }
4049 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4050
4051 // ---- RMS Norm Vec4 ----
4052 dim3 block_rv4(32);
4053 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
4054 check(cudaGetLastError(), "rmsnorm_vec4 launch");
4055 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
4056 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
4057 max_err = 0.0f;
4058 for (int i = 0; i < N * K; i++) {
4059     float err = fabsf(h_y[i] - h_y_ref[i]);
4060     if (err > max_err) max_err = err;
4061 }
4062 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4063
4064 free(h_x); free(h_y); free(h_y_ref);
4065 cudaFree(d_x); cudaFree(d_y);
4066 }
4067
4068
4069 static void test_layer_norm(int N, int K) {
4070
4071     srand(42);
4072     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4073     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4074     for (int i = 0; i < N * K; i++)
4075         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4076     float g = 1.5f, b = 0.3f; // gain and bias
4077
4078     // CPU reference: y = ((x - mean) / std) * g + b
4079     float epsilon = 1e-5f;
4080     for (int n = 0; n < N; n++) {
4081         double sum = 0.0;
4082         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
4083         float mean = (float)sum / (float)K;
4084         double sum_sq = 0.0;
4085         for (int k = 0; k < K; k++) {
4086             float diff = h_x[n * K + k] - mean;
4087             sum_sq += (double)diff * (double)diff;
4088         }
4089         float std = sqrtf((float)sum_sq / (float)K + epsilon);
4090         for (int k = 0; k < K; k++)
4091             h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
4092     }
4093
4094     float *d_x, *d_y;

```

```

4095 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
4096 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
4097 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
4098
4099 dim3 block(128);
4100 dim3 grid(N);
4101 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
4102 check(cudaGetLastError(), "layernorm launch");
4103 check(cudaDeviceSynchronize(), "layernorm sync");
4104
4105 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4106 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
4107
4108 float max_err = 0.0f;
4109 for (int i = 0; i < N * K; i++) {
4110     float err = fabsf(h_y[i] - h_y_ref[i]);
4111     if (err > max_err) max_err = err;
4112 }
4113 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4114
4115 // ---- Layer Norm Vec4 ----
4116 dim3 block_lv4(32);
4117 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
4118 check(cudaGetLastError(), "layernorm_vec4 launch");
4119 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
4120 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
4121 max_err = 0.0f;
4122 for (int i = 0; i < N * K; i++) {
4123     float err = fabsf(h_y[i] - h_y_ref[i]);
4124     if (err > max_err) max_err = err;
4125 }
4126 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4127
4128 free(h_x); free(h_y); free(h_y_ref);
4129 cudaFree(d_x); cudaFree(d_y);
4130 }
4131
4132
4133 static void test_rope(int seq_len, int N) {
4134
4135     int total_pairs = seq_len * N;
4136     int total_elems = total_pairs * 2;
4137     size_t size = (size_t)total_elems * sizeof(float);
4138
4139     float *h_x = (float *)malloc(size);
4140     float *h_y_ref = (float *)malloc(size);
4141
4142     srand(42);
4143     for (int i = 0; i < total_elems; i++)
4144         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4145
4146     // CPU reference: 2D rotation for each pair
4147     for (int idx = 0; idx < total_pairs; idx++) {
4148         int token_pos = idx / N;
4149         int token_idx = idx % N;
4150         float x1 = h_x[idx * 2];
4151         float x2 = h_x[idx * 2 + 1];
4152         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
4153         float angle = (float)token_pos * theta;
4154         float cos_v = cosf(angle);
4155         float sin_v = sinf(angle);
4156         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
4157         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;

```

```

4158 }
4159
4160 float *d_x, *d_y;
4161 check(cudaMalloc(&d_x, size), "rope alloc X");
4162 check(cudaMalloc(&d_y, size), "rope alloc Y");
4163 check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
4164
4165 dim3 block(256);
4166 dim3 grid((total_pairs + 255) / 256);
4167 rope<<<grid, block>>>(d_x, d_y, seq_len, N);
4168 check(cudaGetLastError(), "rope launch");
4169 check(cudaDeviceSynchronize(), "rope sync");
4170
4171 float *h_y = (float *)malloc(size);
4172 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
4173
4174 float max_err = 0.0f;
4175 for (int i = 0; i < total_elems; i++) {
4176     float err = fabsf(h_y[i] - h_y_ref[i]);
4177     if (err > max_err) max_err = err;
4178 }
4179 printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4180
4181 free(h_x);
4182 free(h_y);
4183 free(h_y_ref);
4184 cudaFree(d_x);
4185 cudaFree(d_y);
4186 }
4187
4188
4189 static void test_mat_transpose(int row, int col) {
4190
4191     size_t size_in = (size_t)row * col * sizeof(float);
4192     size_t size_out = (size_t)col * row * sizeof(float);
4193
4194     float *h_x = (float *)malloc(size_in);
4195     float *h_y_ref = (float *)malloc(size_out);
4196
4197     srand(42);
4198     for (int i = 0; i < row * col; i++)
4199         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4200
4201     // CPU reference: y[j][i] = x[i][j] (row-major)
4202     for (int i = 0; i < row; i++)
4203         for (int j = 0; j < col; j++)
4204             h_y_ref[j * row + i] = h_x[i * col + j];
4205
4206     float *d_x, *d_y;
4207     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
4208     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
4209     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
4210
4211     dim3 block(16, 16);
4212     dim3 grid((col + 15) / 16, (row + 15) / 16);
4213     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
4214     check(cudaGetLastError(), "mattrans launch");
4215     check(cudaDeviceSynchronize(), "mattrans sync");
4216
4217     float *h_y = (float *)malloc(size_out);
4218     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
4219
4220     float max_err = 0.0f;

```

```

4221     for (int i = 0; i < col * row; i++) {
4222         float err = fabsf(h_y[i] - h_y_ref[i]);
4223         if (err > max_err) max_err = err;
4224     }
4225     printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4226
4227     free(h_x);
4228     free(h_y);
4229     free(h_y_ref);
4230     cudaFree(d_x);
4231     cudaFree(d_y);
4232 }
4233
4234
4235 static void test_mat_transpose_padded(int row, int col) {
4236
4237     size_t size_in = (size_t)row * col * sizeof(float);
4238     size_t size_out = (size_t)col * row * sizeof(float);
4239
4240     float *h_x = (float *)malloc(size_in);
4241     float *h_y_ref = (float *)malloc(size_out);
4242
4243     srand(42);
4244     for (int i = 0; i < row * col; i++)
4245         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4246
4247     // CPU reference: y[j][i] = x[i][j] (row-major)
4248     for (int i = 0; i < row; i++)
4249         for (int j = 0; j < col; j++)
4250             h_y_ref[j * row + i] = h_x[i * col + j];
4251
4252     float *d_x, *d_y;
4253     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
4254     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
4255     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
4256
4257     dim3 block(16, 16);
4258     dim3 grid((col + 15) / 16, (row + 63) / 64);
4259     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
4260     check(cudaGetLastError(), "mattrans_padded launch");
4261     check(cudaDeviceSynchronize(), "mattrans_padded sync");
4262
4263     float *h_y = (float *)malloc(size_out);
4264     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
4265
4266     float max_err = 0.0f;
4267     for (int i = 0; i < col * row; i++) {
4268         float err = fabsf(h_y[i] - h_y_ref[i]);
4269         if (err > max_err) max_err = err;
4270     }
4271     printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4272
4273     free(h_x);
4274     free(h_y);
4275     free(h_y_ref);
4276     cudaFree(d_x);
4277     cudaFree(d_y);
4278 }
4279
4280
4281 static void test_sgemv(int M, int K) {
4282
4283     srand(42);

```

```

4284 float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
4285 float *h_x = (float *)malloc((size_t)K * sizeof(float));
4286 float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4287 for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4288 for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4289
4290 // CPU reference: y[m] = sum_k A[m,k] * x[k]
4291 for (int m = 0; m < M; m++) {
4292     double sum = 0.0;
4293     for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
4294     h_y_ref[m] = (float)sum;
4295 }
4296
4297 float *d_a, *d_x, *d_y;
4298 check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
4299 check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
4300 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
4301
4302 check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
4303 check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
4304
4305 dim3 block(32, 4);
4306 dim3 grid((M + 3) / 4);
4307 sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
4308 check(cudaGetLastError(), "sgemv launch");
4309 check(cudaDeviceSynchronize(), "sgemv sync");
4310
4311 float *h_y = (float *)malloc((size_t)M * sizeof(float));
4312 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
4313
4314 float max_err = 0.0f;
4315 for (int m = 0; m < M; m++) {
4316     float err = fabsf(h_y[m] - h_y_ref[m]);
4317     if (err > max_err) max_err = err;
4318 }
4319 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4320
4321 // ---- SGEMV K32 ----
4322 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
4323 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
4324 check(cudaGetLastError(), "sgemv_k32 launch");
4325 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
4326
4327 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
4328
4329 max_err = 0.0f;
4330 for (int m = 0; m < M; m++) {
4331     float err = fabsf(h_y[m] - h_y_ref[m]);
4332     if (err > max_err) max_err = err;
4333 }
4334 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4335
4336 // ---- SGEMV K16 ----
4337 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4338 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4339
4340 int K16 = 16;
4341 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
4342 h_x = (float *)malloc((size_t)K16 * sizeof(float));
4343 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4344 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4345 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4346

```

```

4347 for (int m = 0; m < M; m++) {
4348     double sum = 0.0;
4349     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
4350     h_y_ref[m] = (float)sum;
4351 }
4352
4353 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
4354 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
4355 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
4356
4357 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
4358 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
4359
4360 dim3 grid_k16((M + 7) / 8);
4361 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
4362 check(cudaGetLastError(), "sgemv_k16 launch");
4363 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
4364
4365 h_y = (float *)malloc((size_t)M * sizeof(float));
4366 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
4367
4368 max_err = 0.0f;
4369 for (int m = 0; m < M; m++) {
4370     float err = fabsf(h_y[m] - h_y_ref[m]);
4371     if (err > max_err) max_err = err;
4372 }
4373 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4374
4375 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4376 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4377 }
4378
4379
4380 static void test_sgemm(int M, int N, int K) {
4381
4382     size_t size_a = (size_t)M * K * sizeof(float);
4383     size_t size_b = (size_t)K * N * sizeof(float);
4384     size_t size_c = (size_t)M * N * sizeof(float);
4385
4386     float *h_a = (float *)malloc(size_a);
4387     float *h_b = (float *)malloc(size_b);
4388     float *h_c_ref = (float *)malloc(size_c);
4389
4390     srand(42);
4391     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4392     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4393
4394     float *d_a, *d_b, *d_c;
4395     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
4396     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
4397     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
4398
4399     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
4400     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
4401
4402     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
4403     cublasHandle_t handle;
4404     cublasCreate(&handle);
4405     float alpha = 1.0f, beta = 0.0f;
4406     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4407                 &alpha, d_b, N, d_a, K, &beta, d_c, N);
4408     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
4409

```

```

4410 // Kernel
4411 cudaEvent_t start, stop;
4412 cudaEventCreate(&start);
4413 cudaEventCreate(&stop);
4414
4415 dim3 block(32, 32);
4416 dim3 grid((N + 31) / 32, (M + 31) / 32);
4417 sgemmm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
4418 check(cudaGetLastError(), "sgemm launch");
4419 check(cudaDeviceSynchronize(), "sgemm sync");
4420
4421 float *h_c = (float *)malloc(size_c);
4422 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
4423
4424 // Verify
4425 float max_err = 0.0f;
4426 for (int i = 0; i < M * N; i++) {
4427     float err = fabsf(h_c[i] - h_c_ref[i]);
4428     if (err > max_err) max_err = err;
4429 }
4430 printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4431
4432 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
4433
4434 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
4435 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
4436 sgemmm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
4437 check(cudaGetLastError(), "sgemm_vec4 launch");
4438 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
4439
4440 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
4441
4442 max_err = 0.0f;
4443 for (int i = 0; i < M * N; i++) {
4444     float err = fabsf(h_c[i] - h_c_ref[i]);
4445     if (err > max_err) max_err = err;
4446 }
4447 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4448
4449 free(h_a); free(h_b); free(h_c); free(h_c_ref);
4450 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4451 cublasDestroy(handle);
4452 cudaEventDestroy(start);
4453 cudaEventDestroy(stop);
4454 }
4455
4456
4457 static void test_hgemmm_mma(int M, int N, int K) {
4458
4459     size_t size_a = (size_t)M * K * sizeof(half);
4460     size_t size_b = (size_t)K * N * sizeof(half);
4461     size_t size_c = (size_t)M * N * sizeof(half);
4462
4463     half *h_a = (half *)malloc(size_a);
4464     half *h_b = (half *)malloc(size_b);
4465     half *h_c_ref = (half *)malloc(size_c);
4466
4467     srand(42);
4468     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4469     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4470
4471     // Kernel expects B^T [NxK] row-major layout (TN layout convention).
4472     // We store h_b as B [KxN] row-major for cuBLAS, then create B^T for kernel.

```



```

4473 size_t size_b_t = (size_t)N * K * sizeof(half);
4474 half *h_b_t = (half *)malloc(size_b_t);
4475 for (int n = 0; n < N; n++)
4476     for (int k = 0; k < K; k++)
4477         h_b_t[n * K + k] = h_b[k * N + n];
4478
4479 half *d_a, *d_b, *d_b_t, *d_c;
4480 check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
4481 check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
4482 check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
4483 check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
4484
4485 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
4486 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
4487 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
4488
4489 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4490 // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
4491 cublasHandle_t handle;
4492 cublasCreate(&handle);
4493 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4494 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4495             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4496             &beta_h, d_c, CUDA_R_16F, N,
4497             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4498 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
4499
4500 // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
4501 cudaEvent_t start, stop;
4502 cudaEventCreate(&start);
4503 cudaEventCreate(&stop);
4504
4505 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
4506 size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
4507 dim3 block(256);
4508 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4509 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4510 check(cudaGetLastError(), "hgemm launch");
4511 check(cudaDeviceSynchronize(), "hgemm sync");
4512
4513 half *h_c = (half *)malloc(size_c);
4514 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
4515
4516 // Verify
4517 float max_err = 0.0f;
4518 for (int i = 0; i < M * N; i++) {
4519     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4520     if (err > max_err) max_err = err;
4521 }
4522 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4523
4524 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4525 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4526 cublasDestroy(handle);
4527 cudaEventDestroy(start);
4528 cudaEventDestroy(stop);
4529 }
4530
4531
4532 static void test_hgemm_swizzle(int M, int N, int K) {
4533     // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
4534     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
4535     // Kernel: hgemm_mma_stages_tn_swizzle with default template params

```

```

4536 size_t size_a = (size_t)M * K * sizeof(half);
4537 size_t size_b = (size_t)K * N * sizeof(half);
4538 size_t size_c = (size_t)M * N * sizeof(half);
4539
4540 half *h_a = (half *)malloc(size_a);
4541 half *h_b = (half *)malloc(size_b);
4542 half *h_c_ref = (half *)malloc(size_c);
4543
4544
4545 srand(42);
4546 for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4547 for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4548
4549 // Kernel expects B^T [N×K] row-major layout (TN layout convention).
4550 size_t size_b_t = (size_t)N * K * sizeof(half);
4551 half *h_b_t = (half *)malloc(size_b_t);
4552 for (int n = 0; n < N; n++)
4553     for (int k = 0; k < K; k++)
4554         h_b_t[n * K + k] = h_b[k * N + n];
4555
4556 half *d_a, *d_b, *d_b_t, *d_c;
4557 check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
4558 check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
4559 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
4560 check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
4561
4562 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
4563 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
4564 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
4565
4566 // cuBLAS FP16 reference
4567 cublasHandle_t handle;
4568 cublasCreate(&handle);
4569 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4570 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4571              &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4572              &beta_h, d_c, CUDA_R_16F, N,
4573              CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4574 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
4575
4576 // MMA swizzle kernel (default params: K_STAGE=3)
4577 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE_S = 3;
4578 size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
4579 dim3 block(256);
4580 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4581 hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4582 check(cudaGetLastError(), "hgemm_swizzle launch");
4583 check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
4584
4585 half *h_c = (half *)malloc(size_c);
4586 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
4587
4588 // Verify
4589 float max_err = 0.0f;
4590 for (int i = 0; i < M * N; i++) {
4591     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4592     if (err > max_err) max_err = err;
4593 }
4594 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA Swizzle", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4595
4596 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4597 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4598 cublasDestroy(handle);

```

```

4599 }
4600
4601
4602 #if defined(NOTES_V2_ENABLE_CUTE)
4603 static void test_hgemm_cute(int M, int N, int K) {
4604     // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3,3> + kStage=2
4605     // TN layout: C[M×N] = A[M×K] × BT[N×K]
4606     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
4607     // Tile: BM=128, BN=256, BK=32, 128 threads/block
4608
4609     size_t size_a = (size_t)M * K * sizeof(half);
4610     size_t size_b = (size_t)K * N * sizeof(half);
4611     size_t size_c = (size_t)M * N * sizeof(half);
4612
4613     half *h_a = (half *)malloc(size_a);
4614     half *h_b = (half *)malloc(size_b);
4615     half *h_c_ref = (half *)malloc(size_c);
4616
4617     srand(42);
4618     for (int i = 0; i < M * K; i++)
4619         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4620     for (int i = 0; i < K * N; i++)
4621         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4622
4623     // CuTe kernel expects BT [N×K] row-major (TN layout).
4624     size_t size_b_t = (size_t)N * K * sizeof(half);
4625     half *h_b_t = (half *)malloc(size_b_t);
4626     for (int n = 0; n < N; n++)
4627         for (int k = 0; k < K; k++)
4628             h_b_t[n * K + k] = h_b[k * N + n];
4629
4630     half *d_a, *d_b, *d_b_t, *d_c;
4631     check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
4632     check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
4633     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
4634     check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
4635
4636     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
4637     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
4638     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
4639
4640     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4641     cublasHandle_t handle;
4642     cublasCreate(&handle);
4643     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4644     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
4645                 CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
4646                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4647     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");
4648
4649     // CuTe kernel (Stages=2, TN layout)
4650     launch_hgemm_mma_stages_tn_cute<half, 2>(d_a, d_b_t, d_c, M, N, K);
4651     check(cudaGetLastError(), "hgemm_cute launch");
4652     check(cudaDeviceSynchronize(), "hgemm_cute sync");
4653
4654     half *h_c = (half *)malloc(size_c);
4655     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
4656
4657     // Verify
4658     float max_err = 0.0f;
4659     for (int i = 0; i < M * N; i++) {
4660         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4661         if (err > max_err) max_err = err;

```

```

4662 }
4663 printf("| %-35s | %.6e | %-4s |\n", "HGEMM CuTe", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4664
4665 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4666 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4667 cublasDestroy(handle);
4668 }
4669 #endif /* NOTES_V2_ENABLE_CUTE */
4670
4671
4672 #if defined(NOTES_V2_ENABLE_WGMMMA)
4673 static void test_hgemm_wgmma(int M, int N, int K) {
4674     // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
4675     // TN layout: C[M×N] = A[M×K] × BT[N×K]
4676     // Kernel: hgemm_wgmma_stages_tn with default template params
4677
4678     constexpr int BM = 128, BN = 128, BK = 64, K_STAGE = 3, NUM_THREADS = 256;
4679
4680     // M, K must be divisible by tile dims
4681     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
4682         printf("| %-35s | %-12s | %-4s |\n",
4683             "HGEMM WGMMMA", "SKIP", "SKIP");
4684         return;
4685     }
4686
4687     size_t size_a = (size_t)M * K * sizeof(half);
4688     size_t size_b = (size_t)K * N * sizeof(half);
4689     size_t size_c = (size_t)M * N * sizeof(half);
4690
4691     half *h_a = (half *)malloc(size_a);
4692     half *h_b = (half *)malloc(size_b);
4693     half *h_c_ref = (half *)malloc(size_c);
4694
4695     srand(42);
4696     for (int i = 0; i < M * K; i++)
4697         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4698     for (int i = 0; i < K * N; i++)
4699         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4700
4701     // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
4702     size_t size_b_t = (size_t)N * K * sizeof(half);
4703     half *h_b_t = (half *)malloc(size_b_t);
4704     for (int n = 0; n < N; n++)
4705         for (int k = 0; k < K; k++)
4706             h_b_t[n * K + k] = h_b[k * N + n];
4707
4708     half *d_a, *d_b, *d_b_t, *d_c;
4709     check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
4710     check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
4711     check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
4712     check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
4713
4714     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
4715     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
4716         "wgmma H2D B (cuBLAS)");
4717     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
4718         "wgmma H2D B_t (kernel)");
4719
4720     // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
4721     cublasHandle_t handle;
4722     cublasCreate(&handle);
4723     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4724     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,

```

```

4725         CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
4726         CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4727     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
4728           "wgmma D2H ref");
4729
4730     // Create TMA tensor maps for A and B^T
4731     // A[MxK] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
4732     // B^T[NxK] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
4733     CUTensorMap *tma_a =
4734         allocate_and_create_tensor_map(d_a, M / BM, K / BK);
4735     CUTensorMap *tma_b =
4736         allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
4737
4738     // Launch WGMMMA kernel
4739     // BLOCK_SWIZZLE=false → 2D grid, no swizzle
4740     size_t smem_bytes =
4741         K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
4742     cudaFuncSetAttribute(
4743         hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE,
4744         false>,
4745         cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
4746
4747     dim3 block(NUM_THREADS);
4748     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4749     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE, false>
4750         <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
4751     check(cudaGetLastError(), "wgmma launch");
4752     check(cudaDeviceSynchronize(), "wgmma sync");
4753
4754     half *h_c = (half *)malloc(size_c);
4755     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
4756
4757     // Verify
4758     float max_err = 0.0f;
4759     for (int i = 0; i < M * N; i++) {
4760         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4761         if (err > max_err)
4762             max_err = err;
4763     }
4764     printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
4765           max_err < 1.0f ? "PASS" : "FAIL");
4766
4767     free(h_a);
4768     free(h_b);
4769     free(h_b_t);
4770     free(h_c);
4771     free(h_c_ref);
4772     cudaFree(d_a);
4773     cudaFree(d_b);
4774     cudaFree(d_b_t);
4775     cudaFree(d_c);
4776     cudaFree(tma_a);
4777     cudaFree(tma_b);
4778     cublasDestroy(handle);
4779 }
4780 #endif /* NOTES_V2_ENABLE_WGMMMA */
4781
4782
4783 static void test_flash_attn(int seqlen, int head_dim) {
4784     // FlashAttention-2 with split-Q, MMA m16n8k16
4785     int B = 1, H = 8;
4786
4787     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);

```

```

4788 srand(42);
4789 half *h_q = (half *)malloc(sz);
4790 half *h_k = (half *)malloc(sz);
4791 half *h_v = (half *)malloc(sz);
4792 for (int i = 0; i < B * H * seqlen * head_dim; i++) {
4793     h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4794     h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4795     h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4796 }
4797
4798 // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
4799 float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
4800 float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
4801 float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
4802 float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
4803 int count = B * H * seqlen * head_dim;
4804 for (int i = 0; i < count; i++) {
4805     ref_q[i] = __half2float(h_q[i]);
4806     ref_k[i] = __half2float(h_k[i]);
4807     ref_v[i] = __half2float(h_v[i]);
4808 }
4809
4810 float scale = 1.0f / sqrtf((float)head_dim);
4811 for (int bi = 0; bi < B * H; bi++) {
4812     for (int qi = 0; qi < seqlen; qi++) {
4813         // S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale
4814         float smax = -INFINITY;
4815         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
4816         for (int kj = 0; kj < seqlen; kj++) {
4817             float s = 0.0f;
4818             for (int d = 0; d < head_dim; d++)
4819                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
4820                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
4821             S[kj] = s * scale;
4822             if (S[kj] > smax) smax = S[kj];
4823         }
4824         // softmax
4825         double sum_exp = 0.0;
4826         for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
4827         float inv_sum = 1.0f / (float)sum_exp;
4828         // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
4829         for (int d = 0; d < head_dim; d++) {
4830             double o_acc = 0.0;
4831             for (int kj = 0; kj < seqlen; kj++)
4832                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
4833                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
4834             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
4835         }
4836         free(S);
4837     }
4838 }
4839
4840 half *d_q, *d_k, *d_v, *d_o;
4841 check(cudaMalloc(&d_q, sz), "fa alloc Q");
4842 check(cudaMalloc(&d_k, sz), "fa alloc K");
4843 check(cudaMalloc(&d_v, sz), "fa alloc V");
4844 check(cudaMalloc(&d_o, sz), "fa alloc O");
4845 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
4846 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
4847 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
4848
4849 // Template params for kHeadDim=64, kStage=2

```

```

4851 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
4852 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
4853 constexpr int kMmaTileSeqLenQ = 4;
4854 constexpr int kMmaTileSeqLenK = 1;
4855 constexpr int kMmaTileSeqLenP = 4;
4856 constexpr int kMmaTileHeadDimV = 1;
4857 constexpr int kValTileSeqLenQ = 1;
4858 constexpr int kValTileSeqLenK = 8;
4859 constexpr int kValTileSeqLenP = 1;
4860 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
4861
4862 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
4863 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
4864 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
4865                     kStageV * Bc * (kHeadDim + kPadV) +
4866                     Bc * (kHeadDim + kPadV)) * sizeof(half);
4867
4868 dim3 block(128);
4869 dim3 grid((seqlen + Br - 1) / Br, B * H);
4870
4871 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
4872                               kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
4873                               kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
4874                               kStageV, kPadV>
4875     <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
4876 check(cudaGetLastError(), "fa launch");
4877 check(cudaDeviceSynchronize(), "fa sync");
4878
4879 half *h_o = (half *)malloc(sz);
4880 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
4881
4882 float max_err = 0.0f;
4883 for (int i = 0; i < count; i++) {
4884     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
4885     if (err > max_err) max_err = err;
4886 }
4887 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
4888
4889 free(h_q); free(h_k); free(h_v); free(h_o);
4890 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
4891 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
4892 }
4893
4894
4895 int main(int argc, char *argv[]) {
4896 #if defined(NOTES_V2_ENABLE_WGMMMA)
4897     cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
4898 #endif
4899     int M = 1024, N = 1024, K = 1024;
4900     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
4901
4902     printf("=== notes-v2.cu verification harness ===\n");
4903     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
4904     printf("|-----|-----|-----|\n");
4905
4906     test_block_reduce(N);
4907     test_dot(N);
4908     test_relu(1024);
4909     test_elementwise(1024);
4910     test_histogram(1024);
4911     test_merge_attn_states(512, 16, 128);
4912     test_softmax(256);
4913     test_rms_norm(8, 128);

```



```

4914     test_layer_norm(8, 128);
4915     test_rope(8, 128);
4916     test_mat_transpose(256, 256);
4917     test_mat_transpose_padded(256, 256);
4918     test_sgemv(256, 128);
4919     test_sgemm(M, N, K);
4920     test_hgemm_mma(M, N, K);
4921     test_hgemm_swizzle(M, N, K);
4922     #if (defined(NOTES_V2_ENABLE_CUTE))
4923         test_hgemm_cute(M, N, K);
4924     #endif
4925     #if (defined(NOTES_V2_ENABLE_WGMMA))
4926         test_hgemm_wgmma(M, N, K);
4927     #endif
4928     test_flash_attn(1024, 64);
4929
4930     printf("=== All tests done ===\n");
4931     return 0;
4932 }
4933
4934 // =====
4935 // Quick build & run reference
4936 // =====
4937 // # sm_89 单独编译 + 运行:
4938 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
4939 //
4940 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
4941 //     项目内置 third-party/cutlass) :
4942 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
4943 //     -I ../../third-party/cutlass/include \
4944 //     -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
4945 //
4946 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
4947 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
4948 //     -DNOTES_V2_ENABLE_WGMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
4949 //
4950 // # sm_90a + CuTe + WGMMA:
4951 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
4952 //     -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMA \
4953 //     -I ../../third-party/cutlass/include \
4954 //     -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin

```